

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Новосибирский национальный исследовательский государственный
университет»
(Новосибирский государственный университет, НГУ)
Структурное подразделение Новосибирского государственного университета –
Высший колледж информатики Университета (ВКИ НГУ)
КАФЕДРА ИНФОРМАТИКИ

**РАЗРАБОТКА ANDROID-ПРИЛОЖЕНИЯ С РЕАЛИЗАЦИЕЙ
SERIAL PORT PROFILE (SPP) ДЛЯ СБОРА ИНФОРМАЦИИ С
ДАТЧИКОВ В ЭКСПЕРИМЕНТАХ НА СТАТИЧЕСКУЮ
ПРОЧНОСТЬ**

Квалификация программист

Руководитель
к.ф.-м.н., старший научный сотрудник
ИГиЛ СО РАН

Ларичкин А.Ю.

«___»_____ 2025 г.

Студент 4 курса
гр. 107 в

Чистяков А.А.

«___»_____ 2025 г.

Новосибирск

2025

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ, УСЛОВНЫХ ОБОЗНАЧЕНИЙ И ТЕРМИНОВ	4
ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ ВКР	7
1.1 Бизнес-требования	7
1.2 Пользовательские требования	7
1.3 Системные требования	9
1.4 Требования к пользовательскому интерфейсу	11
1.5 План-график выполнения ВКР	12
2 АНАЛИЗ ТРЕБОВАНИЙ И ОПРЕДЕЛЕНИЕ СПЕЦИФИКАЦИЙ	14
2.1 Анализ предметной области и условий применения программного обеспечения	14
2.1.1 Проблематика и ключевые информационные объекты в инженерно-геодезической деятельности	14
2.1.2 Информационные и функциональные потребности пользователей	16
2.1.3 Модель информационного взаимодействия и методы обработки данных	18
2.1.3.1 Аппаратные и программные технологии передачи данных в системе	19
2.1.4 Анализ существующих аналогов и сравнительная характеристика решений	20
2.2 Классы и характеристики пользователей	24
2.3 Функциональные требования	24
2.3.1 Определение функциональных возможностей	25
2.3.2 Описание прецедентов	26
2.4 Нефункциональные требования	29
3 ВЫБОР ПРОГРАММНЫХ СРЕД И СРЕДСТВ РАЗРАБОТКИ	30
3.1 Сравнительный анализ имеющихся возможностей по выбору средств разработки	30
3.2 Характеристика выбранных программных сред и средств	32
4 АЛГОРИТМЫ РЕШЕНИЯ ПОСТАВЛЕННОЙ ЗАДАЧИ	33
4.1 Этапы реализации	33
4.2 Пользовательский интерфейс	33
4.2.1 Описание взаимодействия с пользователем	33
4.2.2 Определение операций пользователей и составление функциональных блоков	35
4.2.3 Проектирование структуры экранов и схемы навигации	35
4.2.4 Разработка дизайна интерфейса	36
4.3 Входные и выходные данные	42

4.4 Реализация используемых методов хранения, обработки и передачи информации об объектах предметной области	42
4.4.1 Методы хранения данных	42
4.4.5 Алгоритмы использования технологий передачи данных	44
4.4.5.1 Межмодульная передача данных	44
4.4.5.2 Передача данных с внешними источниками.....	45
4.5 Описание архитектурного решения	46
4.5.1 Структурная организация программной системы	46
4.5.2 Архитектура программного кода	49
5 ТЕСТИРОВАНИЕ И ОПТИМИЗАЦИЯ	52
5.1 План тестирования.....	52
5.2 Результаты тестирования и оптимизация.....	53
6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ.....	61
ЗАКЛЮЧЕНИЕ	70
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	71
ПРИЛОЖЕНИЯ.....	72
Приложение А	72
Приложение Б.....	74
Приложение В	96

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ, УСЛОВНЫХ ОБОЗНАЧЕНИЙ И ТЕРМИНОВ

SPP (Serial Port Profile) — профиль Bluetooth, предназначенный для эмуляции последовательного порта. Используется для обмена данными между устройствами, как через COM-порт.

API (Application Programming Interface) — интерфейс прикладного программирования. Набор готовых функций, предоставляемых одной программой для использования другой.

CSV (Comma-Separated Values) — формат текстовых файлов для хранения табличных данных, где значения разделяются запятыми (или другими разделителями).

JSON (JavaScript Object Notation) — легковесный формат обмена данными, основанный на структуре «ключ — значение». Широко используется в web и мобильной разработке.

MAC-адрес (Media Access Control address) — уникальный идентификатор сетевого интерфейса устройства. Используется для адресации в сетях уровня канала передачи данных.

TCP/UDP (Transmission Control Protocol / User Datagram Protocol) — два основных протокола транспортного уровня. TCP обеспечивает надёжную доставку данных, UDP — более быструю, но без подтверждения.

UI (User Interface) — пользовательский интерфейс. Совокупность визуальных элементов и способов взаимодействия пользователя с программой.

DB (Database) — база данных, структура для хранения и управления данными.

ВВЕДЕНИЕ

Современные технологии автоматизированного сбора данных играют важную роль в науке и промышленности, обеспечивая точный анализ поведения материалов и конструкций под нагрузкой. Одним из ключевых направлений таких исследований является испытание материалов на статическую прочность.

В мировой практике активно используются различные методы сбора данных в таких экспериментах. Однако традиционные проводные системы имеют ряд недостатков, включая ограниченную мобильность и сложность установки. В связи с этим все большую популярность приобретают беспроводные технологии, обеспечивающие удобство и гибкость при проведении испытаний. Среди них особое место занимает технология Serial Port Profile (SPP), которая позволяет передавать данные с датчиков на удаленные устройства по Bluetooth.

Исследования прочности материалов востребованы во всем мире, и Россия не является исключением. Испытания на статическую прочность проводятся в лабораториях исследовательских институтов, университетов и промышленных предприятий. Однако существующие системы сбора данных часто требуют стационарных комплексов с дорогостоящим оборудованием. В этой связи актуальным становится разработка мобильного решения, способного повысить удобство и оперативность обработки данных.

На региональном уровне, например, в Новосибирской области, данный вопрос также играет важную роль. Регион является крупным центром науки и технологий. Возможность использовать мобильное приложение для сбора данных с датчиков значительно облегчит процесс проведения экспериментов и позволит быстрее анализировать полученные результаты.

Одним из аналогов подобных систем являются промышленные комплексы сбора данных, однако их высокая стоимость и сложность внедрения ограничивают их использование в исследовательских лабораториях и учебных заведениях. Разработка Android-приложения с поддержкой SPP решит эту проблему, предоставляя удобный и доступный инструмент для ученых, инженеров и студентов.

Целью выпускной квалификационной работы является разработка мобильного приложения для обеспечения беспроводного соединения с датчиками, сбора, анализа и экспорта данных.

Для достижения данной цели необходимо решить следующие задачи:

- Провести анализ существующих решений и технологий для организации беспроводного обмена данными через Bluetooth SPP.

- Изучить принципы работы Bluetooth API в Android и его возможности для взаимодействия с внешними устройствами.
- Разработать алгоритм подключения к датчикам, обеспечивающий стабильное соединение и корректный обмен данными.
- Создать удобный и интуитивно понятный пользовательский интерфейс для визуализации полученной информации.
- Реализовать функционал обработки и анализа данных, включая фильтрацию, расчет статистики и построение графиков.
- Разработать систему сохранения данных в удобном формате для последующего использования.
- Реализовать возможность экспорта полученных данных в популярные форматы, такие как CSV и JSON.
- Провести тестирование и отладку приложения для обеспечения надежности работы и удобства использования.

Разработка мобильного приложения на основе Bluetooth SPP повысит эффективность исследований прочности материалов, обеспечив удобство и доступность экспериментов. Внедрение такого решения упростит автоматизированный сбор данных и их интеграцию в научные и промышленные процессы.

1 ПОСТАНОВКА ЗАДАЧИ ВКР

1.1 Бизнес-требования

Научные исследования требуют точных и эффективных инструментов сбора данных, которые могут существенно повлиять на качество получаемых результатов. Разрабатываемое Android-приложение с поддержкой SPP автоматизирует измерения, упрощая работу специалистов и обеспечивая высокую точность данных. Это позволяет существенно снизить время, затрачиваемое на ручные операции, и минимизировать ошибки. Его внедрение позволит решить ряд важных задач, таких как повышение производительности, улучшение качества исследований и оптимизация рабочего процесса.

Прежде всего, автоматизация сбора данных позволит значительно повысить точность измерений. Использование Bluetooth SPP, как указано в работе Спецификации профиля SPP [1], обеспечит стабильное соединение с датчиками, исключая ошибки ручного ввода и задержки, возникающие при традиционных методах регистрации данных.

Кроме того, система повысит удобство работы исследователей и инженеров. Интуитивно понятный интерфейс, возможность мониторинга состояния соединения и управления передачей данных упростят проведение испытаний как в лабораторных, так и в полевых условиях.

Наконец, разработка приложения обеспечит совместимость с современным оборудованием и программными решениями. Использование стандартных технологий Android позволит научным группам и организациям легко интегрировать систему в существующую инфраструктуру без необходимости значительных доработок.

Таким образом, создание мобильного приложения на основе Bluetooth SPP повысит эффективность работы исследовательских коллективов, сократит трудозатраты и ускорит процесс анализа данных.

1.2 Пользовательские требования

Важность систематизированного описания пользовательских требований заключается в обеспечении эффективной коммуникации между заказчиком и разработчиками, что является ключевым фактором в успешной разработке. Четкое формулирование требований помогает удовлетворить организационные цели и

способствует развитию проекта. Для лучшего понимания разрабатываемого программного продукта были сформулированы следующие пользовательские требования:

1. Для исследователей и операторов экспериментов

Подключение к внешним устройствам через Bluetooth с использованием SPP:

- Возможность установить беспроводное соединение с датчиками или другими внешними устройствами.

- Поддержка стандартного протокола SPP для обмена данными.

Просмотр данных и управление соединением:

- Отображение списка доступных Bluetooth-устройств с указанием имени и MAC-адреса.

- Возможность выбора конкретного устройства для установления соединения.

- Мониторинг состояния соединения (например, «подключено» или «отключено»).

- Уведомления об ошибках при установлении соединения или передаче данных.

Ввод, обработка и экспорт данных:

- Окно для отображения и анализа данных, поступающих с датчиков.

- Возможность сохранения и экспорта данных в формате CSV или JSON.

- Графическое представление информации (например, построение графиков).

Интуитивное управление:

- Кнопки для поиска устройств, подключения/отключения, включения/выключения Bluetooth.

- Простота и удобство интерфейса для быстрого доступа к функциям приложения.

2. Для технического персонала и специалистов по тестированию

Совместимость с оборудованием и ОС:

- Работа на устройствах с Android 8.0 (API Level 26) и выше.

- Поддержка Bluetooth-модуля, соответствующего требованиям SPP.

Языковая поддержка:

- Основной язык интерфейса – русский.

- Опциональная возможность переключения на английский язык.

Интеграция с дополнительными инструментами:

- Возможность использования вспомогательных скриптов на Python для анализа данных.

- Совместимость с библиотеками автоматизированного тестирования.

Надёжность и стабильность работы:

- Обеспечение устойчивого соединения с Bluetooth-устройствами.

- Обработка ошибок и исключительных ситуаций для предотвращения сбоев.
- Резервное копирование данных и возможность их восстановления в случае сбоев.

Все требования сформулированы на основе предоставленного технического задания, находящегося в приложении А.

1.3 Системные требования

Для создания функционально полноценной системы, способной обеспечить стабильное соединение, надежный обмен данными с датчиками и удобное взаимодействие с пользователем, определяются следующие системные требования, основанные на рекомендациях Google Developers по Bluetooth API [2]. Они представляют собой высокоуровневое описание подсистем и их конфигурации.

Подсистемы и их функциональность

1. Модуль управления Bluetooth-соединением

Назначение: обеспечивает подключение к внешним устройствам через Bluetooth с использованием протокола SPP [1].

Функциональность:

- Инициализация и управление Bluetooth-адаптером устройства.
- Сканирование доступных Bluetooth-устройств и формирование списка обнаруженных устройств (с указанием имени и MAC-адреса).
- Установка и контроль состояния соединения.
- Обработка ошибок при подключении и передаче данных.

2. Модуль обмена данными

Назначение: обеспечивает передачу текстовой информации между Android-устройством и внешними датчиками.

Функциональность:

- Прием и передача текстовых сообщений через установленное Bluetooth-соединение.
- Кодирование и декодирование передаваемых данных.
- Обеспечение высокой производительности при обмене данными в режиме реального времени.

3. Модуль пользовательского интерфейса (UI)

Назначение: отвечает за взаимодействие пользователя с приложением и управление его основными функциями.

Функциональность:

- Отображение списка доступных Bluetooth-устройств с возможностью выбора устройства для подключения.

- Окно ввода/вывода текстовой информации для обмена сообщениями с датчиками.
- Элементы управления (кнопки) для сканирования, установки/разрыва соединения и активации Bluetooth.

4. Модуль мониторинга и логирования

Назначение: ведет учет состояния системы, логирует действия пользователя и фиксирует возможные ошибки.

Функциональность:

- Отображение статуса соединения (например, «подключено» или «отключено»).
- Логирование событий, связанных с установкой и разрывом соединения, а также передачей данных.

- Обработка и отображение уведомлений об ошибках и исключительных ситуациях.

5. Модуль интеграции с дополнительными инструментами

Назначение: обеспечивает взаимодействие Android-приложения с внешними инструментами, например, со скриптами на Python.

Функциональность:

- Передача полученных данных для дальнейшей обработки на ПК.
- Интеграция с библиотекой PySerial для анализа данных и автоматизированного тестирования.

- Обеспечение совместимости с различными языками программирования.

Организация системы

1. Функциональное разделение

Система разбивается на подсистемы по функциональному признаку, что позволяет:

- Изолировать задачи, связанные с подключением, обменом данными и обработкой информации.

- Облегчить поддержку и расширение функциональности.

2. Конфигурация системы

Приложение разрабатывается на базе Android Studio с использованием Java для основной функциональности. Для специализированных задач применяется Python (например, библиотека PySerial).

Преимущества такой конфигурации:

- Гибкость и масштабируемость.

- Возможность расширения функционала.
- Обеспечение надежного взаимодействия между всеми компонентами системы.

1.4 Требования к пользовательскому интерфейсу

Требования к внешнему виду интерфейса

Графический интерфейс пользователя должен обеспечивать удобное, понятное и интуитивное взаимодействие с разрабатываемым мобильным приложением. Требования к интерфейсу условно делятся на две группы:

Требования к внешнему виду интерфейса;

Внешний вид интерфейса. Интерфейс должен быть простым и эргономичным. Визуальный стиль — современный, с минималистичным оформлением и возможным применением фирменных цветов. Элементы управления (кнопки, поля, индикаторы) должны быть логично размещены, не перегружая экран. Интерфейс должен корректно отображаться на устройствах с различными размерами экранов, включая смартфоны и планшеты. Сообщения (статусы, ошибки, подсказки) должны быть краткими, понятными и содержать полезную информацию для пользователя. Формат ввода данных должен исключать ошибки за счёт ограничений на формат ввода, а вывод должен быть удобен для восприятия и копирования.

Доступ к функциональности. Приложение должно предоставлять пользователю быстрый и интуитивно понятный доступ к основным функциям: сканирование устройств, подключение и отключение, просмотр и отправка данных, мониторинг состояния соединения. Главный экран должен содержать наиболее востребованные функции. Интерактивные элементы управления должны обеспечивать мгновенную визуальную обратную связь (анимации, смена цвета, уведомления). Интерфейс должен адаптироваться под сенсорное управление и различные форм-факторы устройств.

В качестве иллюстрации пользовательского интерфейса в приложении на этапе проектирования составлена структурно-компонентная схема интерфейса, отражающая предполагаемое размещение основных элементов управления и информационных блоков. Схема представлена на рисунке 1 и выполнена в виде наброска от руки, что позволяет наглядно продемонстрировать логическую структуру экранных форм и компоновку интерфейсных компонентов.

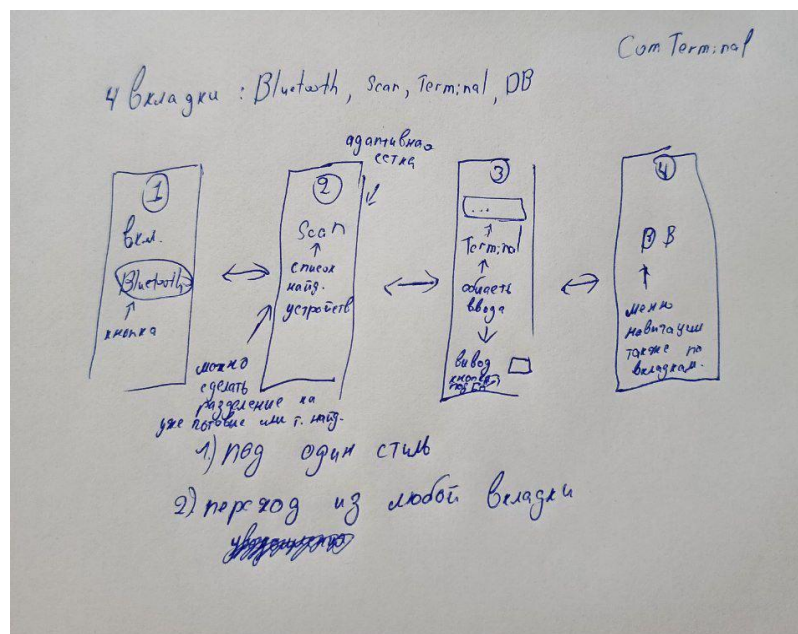


Рисунок 1 – Набросок приложения

1.5 План-график выполнения ВКР

Планирование играет важную роль в разработке программного продукта. Оно помогает выполнить набор задач вовремя и в пределах ограниченного бюджета. Для этого был разработан план действий.

Таблица 1 – План разработки веб-приложения “Разработка Android-приложения с Serial Port Profile (SPP) для сбора данных с датчиков в экспериментах на статическую прочность”

№	Этапы	Наименование работ	Сроки (недели)	Даты
1	Планирование	Составление планов разработки	2	01.10.24 – 16.10.24
		Распределение ролей и обязанностей	1	16.10.24 – 21.10.24
		Составление технического задания	1	21.10.24 – 28.10.24
2	Итерация прототипа	Анализ выполненной работы	1	28.10.24 – 05.11.24
		Сверка с техническим заданием	1	05.11.24 – 11.11.24
		Реализация основного функционала программного обеспечения	6	11.11.24 – 9.12.24
		Разработка необходимых функций	4	9.12.24 – 07.01.25

№	Этапы	Наименование работ	Сроки (недели)	Даты
		Тестирование	1	07.01.25 – 13.01.25
		Согласование с заказчиком	1	13.01.25 – 21.01.25
3	Утверждение прототипа заказчиком	Утверждение прототипа заказчиком, для его доработки до финальной версии	1	21.01.25 – 27.01.25
4	Доработка прототипа до финальной версии	Анализ выполненной работы	2	27.01.25 – 11.02.25
		Сверка с техническим заданием	1	11.02.25 – 17.02.25
		Доработка основного функционала программного обеспечения	6	17.02.25 – 01.04.25
		Доработка необходимых функций	4	01.04.25 – 28.04.25
		Тестирование готового программного обеспечения	4	28.04.23 – 15.05.23
5	Эксплуатация	-	-	-

2 АНАЛИЗ ТРЕБОВАНИЙ И ОПРЕДЕЛЕНИЕ СПЕЦИФИКАЦИЙ

2.1 Анализ предметной области и условий применения программного обеспечения

2.1.1 Проблематика и ключевые информационные объекты в инженерно-геодезической деятельности

Современный мир сталкивается с растущими требованиями к точности, скорости и эффективности в различных областях инженерных и строительных исследований. Строительные проекты, от возведения крупных объектов до разработки инновационных материалов, требуют высокой точности измерений, поскольку это напрямую сказывается на безопасности и долговечности объектов. В условиях быстрого роста урбанизации и изменения климатических условий важно обеспечить максимальную надежность и долговечность инфраструктуры. Для этого необходимо использовать передовые методы научных исследований, которые могут обрабатывать огромные объемы данных в реальном времени, что становится возможным благодаря интеграции высоких технологий.

Современные инженерные исследования нуждаются в инструментах, которые позволяют собирать, обрабатывать и анализировать данные с высокой точностью и минимальными затратами времени. Однако традиционные методы, включающие использование стационарных систем и ручной ввод, начинают не справляться с возрастающими требованиями. В последние десятилетия начинается переход к новым подходам, включающим автоматизацию, мобильные технологии и беспроводную передачу данных. Внедрение таких технологий значительно ускоряет процессы, повышает точность измерений и позволяет минимизировать ошибки, связанные с человеческим фактором.

Одним из ярких примеров таких изменений является внедрение беспроводных технологий, таких как Bluetooth, и мобильных решений, которые начинают активно использоваться в инженерных и строительных исследованиях. Беспроводная передача данных с датчиков на мобильные устройства открывает новые возможности для более гибкого и быстрого сбора информации. Такие системы могут оперативно передавать данные в режиме реального времени, что значительно ускоряет процессы мониторинга и анализа. Более того, с развитием мобильных платформ и приложений инженеры,

исследователи и специалисты получают возможность собирать и обрабатывать данные прямо на месте, не зависимо от стационарных систем.

Этот переход к мобильным и беспроводным технологиям позволяет значительно повысить эффективность работы в таких специализированных областях, как геодезия. Геодезия играет ключевую роль в проектировании и строительстве, обеспечивая точность измерений для всех этапов создания объектов. Однако на практике геодезисты сталкиваются с множеством проблем при сборе данных в полевых условиях. Геодезисты, работающие в сложных и разнообразных природных условиях, ежедневно сталкиваются с тяжёлыми физическими и техническими задачами, которые требуют не только высокой квалификации, но и немалых усилий. Для выполнения точных измерений они часто используют сложные приборы, такие как тахеометры, нивелиры и GPS-устройства, каждый из которых требует особого подхода к настройке и использованию. Эти устройства, хотя и являются высокотехнологичными, часто громоздки, нуждаются в регулярной калибровке и постоянной проверке на точность, что требует значительных временных затрат.

Кроме того, в условиях работы в поле геодезисты сталкиваются с постоянной необходимостью синхронизации данных с несколькими датчиками. Процесс требует внимательности и значительных усилий для корректной настройки каждого устройства. При этом ошибки в настройке оборудования или сбои в его работе могут привести к неправильным измерениям и, как следствие, к искажению всей картины проектных данных, что в свою очередь может повлиять на качество строительных работ. Работа геодезистов осложняется тем, что они часто должны работать в условиях ограниченной инфраструктуры, таких как удалённые и труднодоступные местности. Это требует от них особой мобильности и способности адаптироваться к быстро меняющимся условиям. Ручной ввод данных, часто использующийся в таких условиях, дополнительно увеличивает нагрузку на специалиста, замедляя процесс и увеличивая вероятность ошибок.

Помимо всего прочего, геодезисты часто вынуждены работать с устаревшим оборудованием и программным обеспечением, которое не всегда совместимо с современными технологиями и требует дополнительных усилий для поддержания работоспособности. Сложность заключается не только в самом процессе измерений, но и в необходимости последующей обработки данных, когда полученная информация должна быть тщательно сверена, скорректирована и занесена в компьютерные системы. Проблемы с синхронизацией между различными датчиками и системами могут замедлить

работу, а ошибки в обработке данных, вызванные устаревшими технологиями, могут повлиять на точность результатов.

В таких условиях рабочие процессы в компании становятся менее гибкими, а время, затрачиваемое на сбор, обработку и анализ данных, значительно увеличивается. Каждая задержка или ошибка в передаче данных может привести к серьезным последствиям в проектировании и строительстве, повлиять на сроки и качество выполнения работ. Важно понимать, что данные проблемы могут тормозить развитие компании, снижать её конкурентоспособность и затруднять успешное выполнение проектов.

На рисунке 2 представлена схема, демонстрирующая, как функционирует компания, не использующая программные средства для автоматизации и сбора данных, и как это влияет на процессы работы геодезистов.

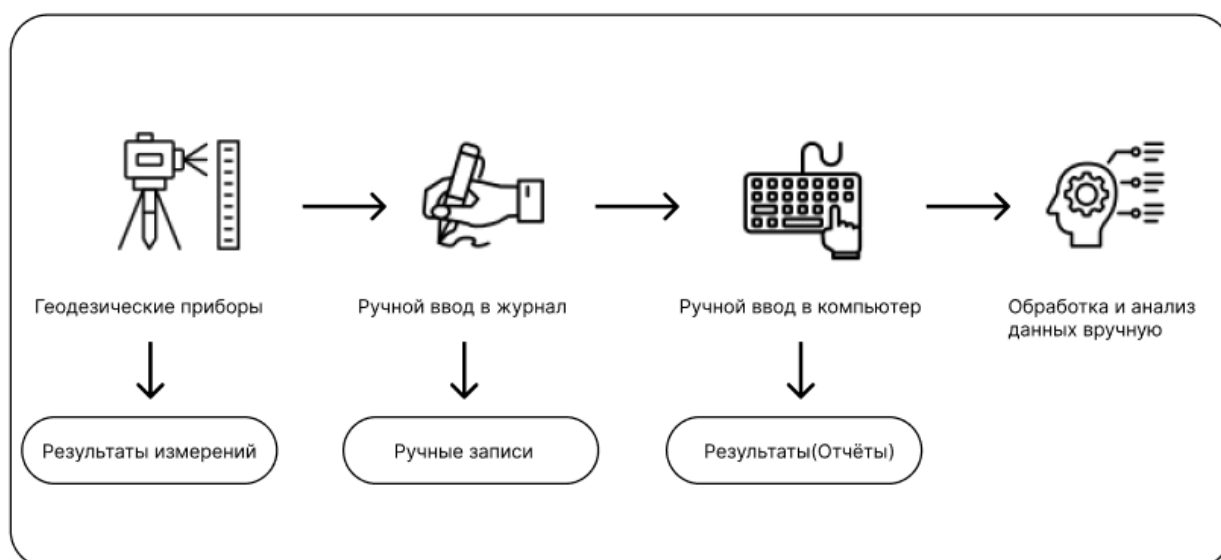


Рисунок 2 – Схема работы специалистов

2.1.2 Информационные и функциональные потребности пользователей

Разрабатываемое программное средство для геодезистов предоставляет широкий спектр функциональных и информационных возможностей для эффективной работы специалистов в данной области. Пользователи получают возможность управлять своими геодезическими данными, что включает в себя отслеживание текущих измерений, анализ точности полученных данных, а также возможность передачи данных с приборов на мобильные устройства в реальном времени для дальнейшей обработки и использования. Эти функции позволяют значительно повысить эффективность работы и удовлетворить профессиональные потребности в точных и своевременных данных.

Одной из ключевых функций приложения является интеграция с геодезическими приборами, что позволяет пользователям получать данные с таких устройств, как тахеометры, нивелиры и GPS. Эти данные автоматически передаются в приложение через Bluetooth или другие беспроводные технологии, устраняя необходимость в ручной передаче и минимизируя возможные ошибки, связанные с человеческим фактором. Полученные данные включают координаты, высоты, углы, а также другие параметры, которые необходимы для точных расчетов и создания планов, чертежей или 3D-моделей.

Для удобства пользователей приложение включает в себя возможность хранения и обработки данных, полученных в ходе полевых работ. Все результаты измерений сохраняются в базе данных, что позволяет в любой момент времени получить доступ к необходимым данным для дальнейшей работы. Пользователи могут просматривать и экспортировать информацию, что облегчает взаимодействие с другими специалистами и заказчиками, а также позволяет быстро генерировать отчёты и схемы.

Важной функцией является управление настройками и параметрами приборов. Геодезисты могут быстро настраивать соединение с приборами, отслеживать их состояние и проверять точность работы в реальном времени. Также пользователи могут получать уведомления о возможных ошибках или неисправностях оборудования, что позволяет своевременно устранять проблемы и предотвращать возможные сбои в процессе работы.

Не менее важной функцией является история измерений и данных. В приложении предусмотрена возможность отслеживания всех выполненных операций, включая сохранение и обработку полученных результатов. Это позволяет специалистам не только держать под контролем текущие данные, но и анализировать изменения, тренды и отклонения в работе приборов и измерений. История транзакций позволяет производить анализ выполненных полевых работ, выявлять закономерности и улучшать процесс измерений в будущем.

Для обеспечения безопасности данных и работы с геодезической информацией приложение включает в себя функции управления персональными данными. Пользователи могут обновлять свои контактные данные, пароли и настройки безопасности, что позволяет обеспечить защиту от несанкционированного доступа. Кроме того, приложение предоставляет возможность резервного копирования данных, что исключает потерю важной информации в случае сбоев в системе.

Функциональность аналитики и отчетности в разрабатываемом приложении позволяет геодезистам получать полное представление о своих данных и результатах. С помощью аналитических инструментов пользователи могут анализировать точность измерений, сравнивать различные этапы работ, а также получать рекомендации по

оптимизации процессов. Эти инструменты помогают не только повысить качество работы, но и принимать более обоснованные решения при планировании будущих проектов и бюджетировании. Для более детального представления о том, как взаимодействуют информационные объекты в рамках функционирования системы, рассмотрим схему, представленную на рисунке 3, которая демонстрирует процесс работы организации с программным средством.

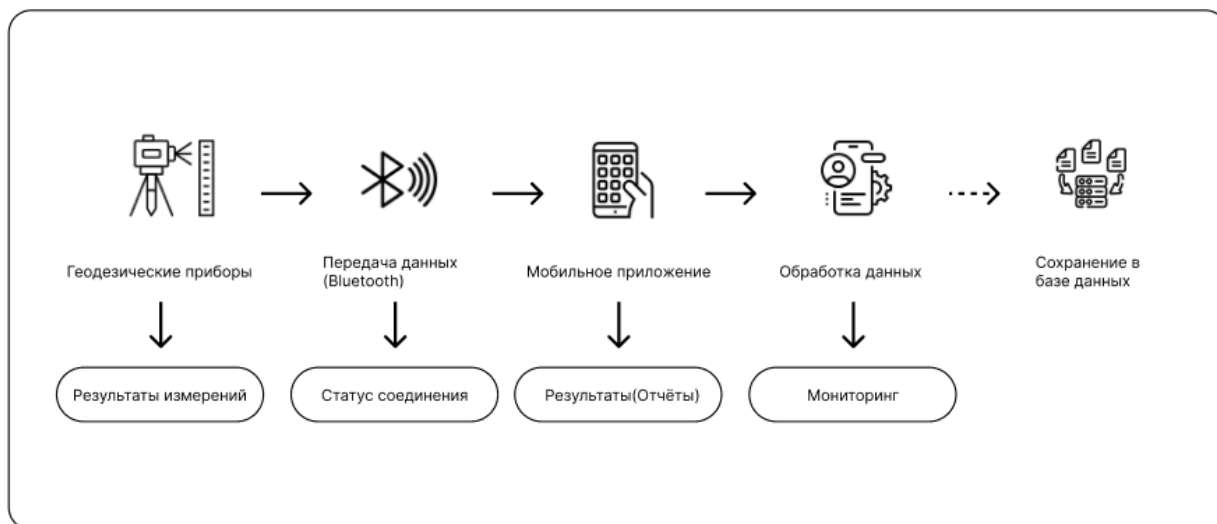


Рисунок 3 – Схема работы системы с ПК

2.1.3 Модель информационного взаимодействия и методы обработки данных

Разрабатываемое программное средство для автоматизации геодезических работ предоставляет комплекс инструментов для сбора, обработки, анализа и хранения данных, полученных с геодезических приборов. Система ориентирована на повышение точности измерений, снижение временных затрат и минимизацию ошибок, связанных с ручным вводом данных.

Предметная область данного программного средства включает в себя следующие основные информационные объекты:

- **Пользователь** — содержит информацию о специалисте, выполняющем геодезические измерения, его роли в системе, а также настройки, связанные с обработкой данных. Данный объект обеспечивает управление доступом к системе и персонализацию функционала.
- **Геодезическое оборудование** — справочник, содержащий сведения о тахеометрах, нивелирах, GPS-устройствах и других приборах. Включает технические характеристики, настройки подключения и методы калибровки.
- **Измерение** — объект, содержащий данные, полученные в ходе геодезических работ. Включает координаты, высоты, углы, временные метки и другие параметры.

Система автоматически привязывает измерения к соответствующему оборудованию и пользователю.

- **Проект** — объединяет набор измерений, связанных с конкретным строительным или исследовательским объектом. Содержит информацию о месте проведения работ, их статусе и назначении.

- Методы работы с данными в разрабатываемой системе включают:

- **Автоматический импорт данных** — измерения поступают с геодезического оборудования через Bluetooth-соединение и сохраняются в базе данных без необходимости ручного ввода.

- **Обработка данных в реальном времени** — система выполняет проверку данных на корректность, исключает дублирование и предлагает рекомендации по настройке оборудования.

- **Фильтрация и анализ** — пользователи могут просматривать измерения, а также анализировать расхождения между полученными и расчетными значениями.

- **Хранение и архивирование** — вся информация сохраняется в базе данных и может быть экспортирована в CSV-файл для интеграции с другими системами или передачи информации третьей стороне.

- **Визуализация данных** — программное средство предоставляет графические представления измерений, карты высот и схемы объектов для удобного анализа.

Информационные объекты взаимодействуют между собой для обеспечения целостного рабочего процесса. Например, пользователь подключается к оборудованию, выполняет измерения, которые затем привязываются к проекту. Интеграция всех процессов в единой системе позволяет значительно повысить точность и скорость выполнения геодезических работ.

2.1.3.1 Аппаратные и программные технологии передачи данных в системе

В рамках разработки Android-приложения используются технологии передачи информации, обеспечивающие устойчивое подключение к внешним устройствам через Bluetooth. Основной механизм взаимодействия основан на использовании Bluetooth API, входящего в состав Android SDK.

Алгоритм взаимодействия с внешними сервисами

1. Сканирование доступных устройств:

- Приложение запрашивает у системы список Bluetooth-устройств, находящихся в зоне досягаемости.

- Отображается список с именами и MAC-адресами найденных устройств.
2. Подключение к выбранному устройству:
- После выбора пользователем устройства устанавливается соединение через Serial Port Profile.
 - Используется сокетное соединение `BluetoothSocket` для обмена данными.
 - В случае успешного подключения пользователь получает уведомление о статусе соединения.
3. Обмен данными:
- Данные передаются в виде текстовых сообщений через поток ввода/вывода, используя `InputStream` и `OutputStream`.
 - Обрабатываются возможные ошибки при передаче данных, например, разрыв соединения или неправильный формат передаваемых данных.
4. Мониторинг состояния соединения:
- В случае отключения устройства или потери соединения приложение сообщает об этом пользователю и предлагает повторное подключение.

2.1.4 Анализ существующих аналогов и сравнительная характеристика решений

Исследуя предметную область, были изучены программные решения, направленные на выполнение схожих задач. В ходе анализа рассмотрены существующие аналоги, их функциональные возможности, достоинства и недостатки.

1. “WiFi Terminal” (https://play.google.com/store/apps/details?id=com.oytechnology.wifi_terminal&hl=ru) – мобильное приложение для Android, которое позволяет устанавливать соединение с удалёнными устройствами через Wi-Fi, используя TCP/UDP-протоколы. Оно предназначено для отправки и приёма данных между смартфоном и различными сетевыми устройствами, такими как микроконтроллеры, серверы или другие компьютеры.

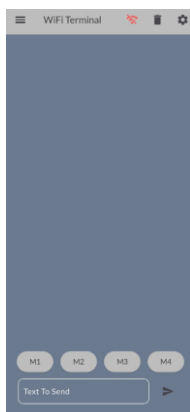


Рисунок 4 – Главная страница мобильного приложения “WiFi Terminal”

Достоинства:

- 1) Гибкость подключения – поддерживает работу по Wi-Fi через TCP и UDP, позволяя подключаться к различным устройствам.
- 2) Высокая скорость передачи данных – Wi-Fi обеспечивает более быструю передачу данных по сравнению с Bluetooth

Недостатки:

- 1) Зависимость от сети Wi-Fi – требуется предварительная настройка сети или создание точки доступа, что не всегда удобно в полевых условиях.
- 2) Сложность настройки – пользователю необходимо вручную вводить IP-адрес и порт для соединения, в отличие от Bluetooth, где подключение происходит автоматически.

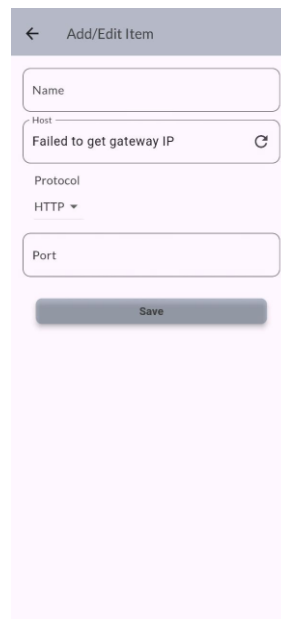


Рисунок 5 – Страница подключения приложения “WiFi Terminal”

3) Проблемы с безопасностью – если Wi-Fi-сеть не защищена паролем, возможен перехват данных или несанкционированный доступ.

4) Ограничения в Android – новые версии Android накладывают ограничения на работу Wi-Fi в фоновом режиме, что может вызывать разрывы соединения.

2. “USB Serial Monitor” (<https://play.google.com/store/apps/details?id=com.keuwl.usbserialmonitor&hl=ru>) – мобильное приложение для Android, предназначенное для передачи данных между устройствами через USB-кабель. Оно позволяет устанавливать соединение с микроконтроллерами, датчиками, диагностическим оборудованием и другими устройствами, поддерживающими последовательный интерфейс (UART, RS232, RS485).



Рисунок 6 – Главная страница мобильного приложения “USB Serial Monitor”

Достоинства:

- 1) Стабильность соединения – в отличие от беспроводных технологий (Wi-Fi, Bluetooth), USB не подвержен радиопомехам и не теряет данные из-за слабого сигнала.
- 2) Не требует беспроводных модулей – устройства могут взаимодействовать напрямую без необходимости поддержки Bluetooth или Wi-Fi.
- 3) Безопасность передачи данных – USB не передает данные по открытому радиоканалу, что исключает риски перехвата.

Недостатки:

- 1) Ограниченная мобильность – требуется физическое подключение, что неудобно для портативных устройств и работы в полевых условиях.
- 2) Зависимость от кабелей и разъёмов – возможны разрывы связи при повреждении кабеля, износе разъёма или некачественном соединении.
- 3) Не все устройства поддерживают USB-OTG – смартфоны и планшеты могут не поддерживать работу с USB-периферией без специальных драйверов или root-доступа.
- 4) Неудобство в многозадачности – в отличие от Bluetooth и Wi-Fi, USB-соединение занимает разъём, что может мешать использованию других периферийных устройств.
- 5) Ограниченная длина кабеля – стандартные USB-кабели имеют длину до 5 метров (для USB 2.0), что ограничивает дальность работы.
- 6) Повышенный риск физического повреждения – при неосторожном использовании кабель или разъём могут сломаться, расшататься или выйти из строя.

В таблице 2 представлено сравнение аналогов с разрабатываемым приложением.

Таблица 2 – Сравнение аналогов и разрабатываемого проекта

Критерий	WiFi Terminal (TCP/UDP)	USB Serial (OTG)	Разрабатываемый проект (Bluetooth SPP)
Тип соединения	Беспроводное (Wi-Fi)	Проводное (USB)	Беспроводное (Bluetooth)
Зависимость от внешней сети	Требуется Wi-Fi сети	Требуется физическое подключение	Не требует интернета и кабелей
Автономность	Ограничена дальностью Wi-Fi (10- 30 метров)	Ограничена длиной кабеля	До 100 метров
Стабильность соединения	Возможны помехи и обрывы	Надёжное, но подвержено механическим повреждениям	Стабильное, помехоустойчивое
Настройка подключения	Вручную (IP-адрес, порт)	Вручную (драйверы, OTG-кабель)	Автоматический поиск и подключение
Энергоэффек- тивность	Высокое энергопотребление	Требуется питания по USB	Низкое энергопотребление
Безопасность данных	Возможен перехват в открытых сетях	Высокая (кабельное соединение)	Высокая (шифрование Bluetooth)
Простота использования	Требуется настроек сети	Требуется физического подключения	Интуитивно понятное подключение
Применение в производствен- ных и полевых условиях	Неудобно (нужна сеть)	Неудобно (ограничение длины кабеля)	Удобно (беспроводная связь без интернета)

Вывод: Разрабатываемый проект на основе Bluetooth SPP представляет собой оптимальное решение для беспроводного обмена данными в производственных и полевых условиях. Он сочетает в себе преимущества Wi-Fi и USB-подключений, устраняя их основные недостатки. Отсутствие зависимости от интернета и физического подключения делает систему по-настоящему мобильной и автономной. Автоматическая настройка подключения упрощает использование, а низкое энергопотребление повышает эффективность при длительной работе. Высокая стабильность и помехоустойчивость Bluetooth обеспечивают надёжную передачу данных, а встроенные механизмы

шифрования — безопасность информации. Всё это делает разработанный проект наиболее предпочтительным выбором для задач, где требуется стабильное, безопасное и простое в использовании соединение без инфраструктурных ограничений.

2.2 Классы и характеристики пользователей

В приложении ComTerminal пользователь представляет собой квалифицированного специалиста, обладающего базовыми знаниями в области цифровых технологий и принципов беспроводной связи. Он эффективно использует возможности приложения для подключения к внешним устройствам по протоколу Bluetooth SPP, обеспечивая стабильный обмен данными в режиме реального времени. Пользователь уверенно ориентируется в интерфейсе: выбирает необходимое устройство, отслеживает статус соединения, обрабатывает входящие и исходящие сообщения. Хотя глубоких технических знаний не требуется, пользователь, как правило, имеет практический опыт работы с измерительным, диагностическим или другим специализированным оборудованием, что позволяет ему применять функционал ComTerminal для решения профессиональных задач в инженерной, производственной или исследовательской среде.

2.3 Функциональные требования

Программное средство предоставляет широкий спектр функциональных возможностей, обеспечивающие эффективную работу с Bluetooth-соединениями через профиль (SPP). Вот описание ключевых возможностей программы:

Персонализированный интерфейс. Приложение предоставляет пользователю интуитивно понятный интерфейс с элементами управления для взаимодействия с Bluetooth-устройствами. Пользователь может легко выбрать доступное устройство для подключения, а также контролировать состояние соединения. Интерфейс также позволяет отображать информацию о подключенных устройствах, включая имя и MAC-адрес.

Управление соединением. Программа предоставляет пользователю возможность устанавливать и поддерживать соединение с внешними Bluetooth-устройствами через SPP. Это включает автоматическое обнаружение доступных устройств, отображение текущего состояния соединения (подключено/отключено) и обработку ошибок при попытке подключения или передачи данных. В случае проблем с соединением или потерей связи приложение уведомляет пользователя и предоставляет возможность восстановления соединения.

Работа с текстовой информацией. Приложение включает окно для ввода и вывода текстовой информации, что позволяет пользователю передавать и получать данные через Bluetooth-соединение. Все полученные данные отображаются в приложении, а введенная информация передается на выбранное устройство. Это позволяет эффективно обмениваться данными в реальном времени.

Поддержка устройства и операционной системы. Приложение совместимо с устройствами на операционной системе Android версии 8.0 (API Level 26) или выше, поддерживающими Bluetooth. Оно также должно работать с различными моделями Android-устройств, обеспечивая универсальность использования.

2.3.1 Определение функциональных возможностей

На основе вариантов использования ПС, представленных в UML диаграмме, определяются функциональные возможности системы для пользователя.

Возможности пользователя:

- Управление подключениями. Пользователь должен иметь возможность включать Bluetooth, а также при необходимости включать дополнительные разрешения, такие как доступ к геопозиции и файлам.
- Поиск и выбор устройств. Пользователь может сканировать доступные устройства Bluetooth для нового подключения или выбирать уже подключённые устройства из списка.
- Управление соединениями. Пользователь может разрывать соединение с устройством с помощью специальной кнопки.
- Отправка сообщений в терминал. Пользователь должен иметь возможность вводить сообщения в терминал и отправлять их на подключённое устройство.
- Сохранение данных в базе. Пользователь может добавлять полученные ответы из терминала в локальную базу данных.
- Работа с базой данных. Пользователь должен иметь возможность загружать данные из базы, удалять отдельные записи или полностью очищать базу данных.
- Экспорт данных. Пользователь может отправлять сохранённую информацию в Dropbox для хранения и дальнейшего использования.

Функциональные возможности пользователей отображены на рисунке 7.



Рисунок 7 – UML-диаграмма прецедентов

2.3.2 Описание прецедентов

Прецеденты представляют собой конкретные сценарии взаимодействия пользователя с программной системой, описывающие ее функциональность. Для более наглядного представления прецедентов и их успешных сценариев представлены таблицы 3-12.

Таблица 3 – Описание прецедента “Включение Bluetooth”

Исполнитель	Пользователь
Цель	Включение Bluetooth и необходимых разрешений.
Основной успешный сценарий	Пользователь выбирает опцию "Включить Bluetooth". Система запрашивает разрешение (если требуется) и активирует Bluetooth.
Ссылки	Bluetooth успешно включён.

Таблица 4 – Описание прецедента “Сканирование устройств”

Исполнитель	Пользователь
Цель	Поиск новых Bluetooth-устройств.
Основной успешный сценарий	Пользователь выбирает опцию "Сканировать устройства". Система запускает процесс поиска и отображает список найденных устройств.
Ссылки	Найденные устройства отображены в списке.

Таблица 5 – Описание прецедента “ Выбор подключённого устройства”

Исполнитель	Пользователь
Цель	Подключение к ранее использованному устройству.
Основной успешный сценарий	Пользователь открывает список уже подключённых устройств и выбирает нужное. Система устанавливает соединение с выбранным устройством.
Ссылки	Соединение с устройством установлено.

Таблица 6 – Описание прецедента “ Разрыв соединения”

Исполнитель	Пользователь
Цель	Отключение от Bluetooth-устройства.
Основной успешный сценарий	Пользователь открывает список уже подключённых устройств и выбирает нужное. Система устанавливает соединение с выбранным устройством.
Ссылки	Соединение с устройством установлено.

Таблица 7 – Описание прецедента “ Отправка сообщения в терминал”

Исполнитель	Пользователь
Цель	Отправка сообщения на подключённое устройство через терминал.
Основной успешный сценарий	Пользователь вводит текст сообщения в терминале и нажимает "Отправить". Система передаёт данные на подключённое устройство через Bluetooth.
Ссылки	Сообщение успешно отправлено.

Таблица 8 – Описание прецедента “ Добавление ответа из терминала в БД”

Исполнитель	Пользователь
Цель	Сохранение полученных данных в локальную базу данных.
Основной успешный сценарий	После получения ответа в терминале пользователь нажимает "Сохранить в БД". Система записывает данные в локальную SQLite-базу через Room.
Ссылки	Данные успешно сохранены в базе.

Таблица 9 – Описание прецедента “ Загрузка данных из БД”

Исполнитель	Пользователь
Цель	Получение сохранённых данных из базы.
Основной успешный сценарий	Пользователь нажимает "Загрузить базу". Система извлекает данные из локальной базы SQLite и отображает их в интерфейсе.
Ссылки	Данные успешно загружены и отображены.

Таблица 10 – Описание прецедента “ Удаление данных из БД”

Исполнитель	Пользователь
Цель	Удаление конкретной записи из базы данных.
Основной успешный сценарий	Пользователь выбирает запись и нажимает "Удалить". Система удаляет выбранную запись из базы.
Ссылки	Запись успешно удалена.

Таблица 11 – Описание прецедента “ Очистка базы данных”

Исполнитель	Пользователь
Цель	Полное удаление всех данных из базы.
Основной успешный сценарий	Пользователь нажимает кнопку "Очистить базу". Система удаляет все записи из локальной базы SQLite.
Ссылки	База данных успешно очищена.

Таблица 12 – Описание прецедента “ Отправка информации в Dropbox”

Исполнитель	Пользователь
Цель	Экспорт данных из базы в Dropbox.
Основной успешный сценарий	Пользователь выбирает данные и нажимает "Отправить в Dropbox". Система формирует файл и загружает его в облачное хранилище.
Ссылки	Данные успешно загружены в Dropbox.

2.4 Нефункциональные требования

Разрабатываемое в рамках выпускной квалификационной работы мобильное приложение для оптимальной работы должно соответствовать следующим характеристикам:

- Смартфон: операционная система Android версии 8.0 (API Level 26) или выше, с поддержкой Bluetooth.
- Модуль Bluetooth: устройство должно поддерживать стандарт Bluetooth 4.0 или выше для корректного соединения.
- Минимальные требования к производительности устройства: не менее 2 ГБ оперативной памяти, процессор с тактовой частотой от 2 ГГц.
- Свободное место в памяти устройства: не менее 50 МБ для установки приложения и хранения временных данных.

Программное средство в виде мобильного приложения должно корректно функционировать на устройствах с сенсорным экраном, а также обеспечивать удобное взаимодействие с пользователем при помощи стандартных элементов управления Android.

Система должна быть устойчивой к сбоям и обеспечивать автоматическое восстановление соединения с Bluetooth-устройствами в случае кратковременного разрыва связи.

Для работы с Dropbox требуется интернет-соединение со скоростью не менее 512 кбит/сек, как указано в Google Codelabs [3].

Приложение должно запрашивать у пользователя необходимые разрешения (доступ к Bluetooth, геопозиции и файловому хранилищу) перед выполнением соответствующих операций, а также обеспечивать защиту пользовательских данных при работе с локальной базой данных.

Система должна быть надёжной, минимизировать количество сбоев и предусматривать механизмы их обработки для удобства пользователя.

3 ВЫБОР ПРОГРАММНЫХ СРЕД И СРЕДСТВ РАЗРАБОТКИ

3.1 Сравнительный анализ имеющихся возможностей по выбору средств разработки

При выборе языка программирования и технологий для разработки мобильного приложения "ComTerminal" был проведён анализ нескольких вариантов. Основным языком разработки был выбран Java благодаря своей кроссплатформенности, широкими возможностям для работы с многозадачностью и большому количеству готовых библиотек и фреймворков, что ускоряет разработку и улучшает поддержку приложения.

Также был рассмотрен Kotlin, который является официальным языком для Android-разработки и предлагает лаконичный синтаксис, более безопасную работу с null-значениями и улучшенную совместимость с Java. Python использовался для написания вспомогательных скриптов, для тестирования и обработки данных. В таблице 13 представлен сравнительный анализ языков программирования.

Таблица 13 – Сравнительный анализ языков программирования

Язык программирования	Преимущества	Недостатки	Сложность применения
Java	Высокая производительность, стабильность, кроссплатформенность	Большой объем кода, необходимость явного управления памятью	Высокая
Kotlin	Краткий и читаемый код, безопасная работа с null, официальная поддержка Google	Требуется изучения для разработчиков, привыкших к Java	Средняя
Python	Удобен для написания вспомогательных скриптов и работы с данными	Ограниченные возможности для Android-разработки	Низкая

Для работы с локальной базой данных в мобильном приложении были рассмотрены несколько технологий, включая SQLite, Room и Realm.

SQLite является встроенной реляционной базой данных Android, обеспечивая высокую производительность и стабильность. Однако, работа с ним требует написания SQL-запросов вручную и дополнительного управления соединениями.

Room – это абстрактный слой над SQLite, который предоставляет удобный API, совместимый с архитектурными компонентами Android. Он упрощает работу с базой данных, поддерживает проверки на этапе компиляции и позволяет работать с объектами вместо написания SQL-запросов вручную.

Realm предлагает альтернативный подход к хранению данных, обеспечивая быструю работу и автоматическую синхронизацию, но он не является стандартным решением для Android и требует дополнительных зависимостей. В таблице 14 представлен сравнительный анализ технологий для работы с базой данных.

Таблица 14 – Сравнительный анализ технологий для работы с базой данных

Технология	Преимущества	Недостатки	Сложность применения
SQLite	Встроенная поддержка в Android, высокая производительность	Необходимость писать SQL-запросы вручную, отсутствие встроенных проверок	Высокая
Room	Упрощенная работа с базой данных, поддержка LiveData и Flow, автоматическая валидация SQL-запросов	Небольшая сложность в настройке аннотаций	Средняя
Realm	Высокая скорость работы, автоматическая синхронизация	Требует дополнительной зависимости, ограниченная поддержка сложных SQL-запросов	Высокая

Исходя из проведённого анализа, для разработки мобильного приложения "ComTerminal" был выбран язык программирования Java, так как он обеспечивает высокую производительность, стабильность и широкую поддержку Android API. Kotlin был рассмотрен как альтернатива, но в данном случае Java оказался более предпочтительным из-за удобной интеграции с существующими библиотеками и инструментами Android. Python используется для вспомогательных задач, таких как тестирование и обработка данных.

Для работы с локальной базой данных было выбрано Room, так как он предоставляет удобный API для работы с SQLite, автоматическую валидацию SQL-запросов на этапе компиляции и поддержку архитектурных компонентов Android. Это решение значительно упрощает работу с базой данных, улучшает читаемость кода и минимизирует вероятность ошибок.

3.2 Характеристика выбранных программных сред и средств

При разработке мобильного приложения "ComTerminal" выбраны следующие технологии:

- Java – основной язык программирования приложения. Java обеспечивает высокую производительность, стабильность и широкую поддержку Android API, что делает его оптимальным выбором для разработки мобильных приложений.
- Android Studio – интегрированная среда разработки (IDE) для Android-приложений, предоставляющая удобные инструменты для написания, тестирования и отладки кода. Она поддерживает эмуляторы устройств, автоматизированное управление зависимостями и интеграцию с Android SDK.
- Python – используется для разработки вспомогательных скриптов, анализа данных и проведения тестирования. Благодаря своему простому синтаксису, он идеально подходит для работы с большими объемами данных и интеграции с внешними сервисами.
- Visual Studio Code (VS Code) – легкая и мощная среда разработки, которая используется для написания и отладки Python-скриптов. Она поддерживает широкий спектр расширений, упрощающих процесс разработки и тестирования.
- Room (на базе SQLite) – библиотека для работы с локальной базой данных в Android-приложении. Она обеспечивает удобный API для взаимодействия с базой, автоматическую валидацию SQL-запросов и поддержку LiveData и Flow, что значительно упрощает управление данными.

Эти технологии были выбраны с учётом их производительности, стабильности и широкого сообщества разработчиков, что делает их оптимальными для успешной реализации мобильного приложения "ComTerminal".

4 АЛГОРИТМЫ РЕШЕНИЯ ПОСТАВЛЕННОЙ ЗАДАЧИ

4.1 Этапы реализации

Разработка мобильного приложения ComTerminal осуществляется поэтапно, с акцентом на реализацию как клиентской, так и внутренней логики приложения. На первом этапе основное внимание уделяется клиентской части — пользовательскому интерфейсу и взаимодействию. Пользователь имеет возможность устанавливать соединение с внешними устройствами по беспроводному каналу, выбирать доступные устройства, отправлять и получать сообщения в режиме реального времени. Интерфейс приложения включает в себя экраны для управления подключениями, отображения текстовых сообщений, журналирования событий и работы с сохранённой информацией. Простой и понятный дизайн делает приложение удобным даже для специалистов, не обладающих глубокими техническими знаниями, но активно применяющих цифровые инструменты в своей деятельности.

Следующим важным этапом является реализация внутренней части приложения, отвечающей за обработку, хранение и управление данными. Внутри приложения предусмотрена система хранения информации о подключениях, сообщениях и действиях пользователя. Эта система позволяет вести учёт взаимодействий, упрощает анализ и обеспечивает возможность повторного доступа к сохранённым данным. Кроме того, реализована возможность выборочного экспорта информации, что делает приложение универсальным инструментом для различных прикладных задач.

Таким образом, процесс разработки ComTerminal включает в себя как проектирование интерфейса и логики взаимодействия с пользователем, так и реализацию надёжной внутренней системы хранения и управления данными. Каждый этап сопровождается тестированием и отладкой, что обеспечивает стабильность и функциональность конечного программного решения.

4.2 Пользовательский интерфейс

4.2.1 Описание взаимодействия с пользователем

Передача данных, выполнение команд и обработка полученной информации являются ключевыми аспектами работы приложения ComTerminal. В рамках разработки были определены основные сценарии взаимодействия пользователя с системой.

Процесс работы с мобильным приложением:

Пользователь запускает приложение и включает Bluetooth на своем устройстве, чтобы обеспечить возможность взаимодействия с внешними приборами.

Приложение выполняет поиск доступных Bluetooth-устройств, отображая список найденных приборов, к которым можно подключиться.

Пользователь также может выбрать одно из ранее запомненных устройств, с которыми уже устанавливал соединение, что ускоряет процесс подключения и делает работу с приложением удобнее.

После выбора нужного устройства устанавливается соединение, что позволяет пользователю отправлять команды и получать ответы.

Пользователь вводит команду, которую необходимо передать подключенному прибору, и отправляет ее через интерфейс приложения.

Приложение передает команду на устройство и ожидает ответа. Как только ответ получен, он отображается на экране в удобном формате.

Чтобы сохранить важные данные, пользователь может занести полученный ответ в локальную базу данных (Room). Это позволяет просматривать историю взаимодействий с приборами и анализировать переданные данные.

При необходимости пользователь может загружать сохраненные данные из базы, чтобы просмотреть ранее полученные ответы или использовать их для последующего анализа.

Для удобства передачи информации предусмотрена возможность выгрузки данных в Dropbox. Пользователь может выбрать нужные записи и отправить их в облачное хранилище для дальнейшего использования или резервного копирования.

На рисунке 8 представлена схема взаимодействия пользователя с приложением в формате диаграммы последовательности, выполненной на языке UML.

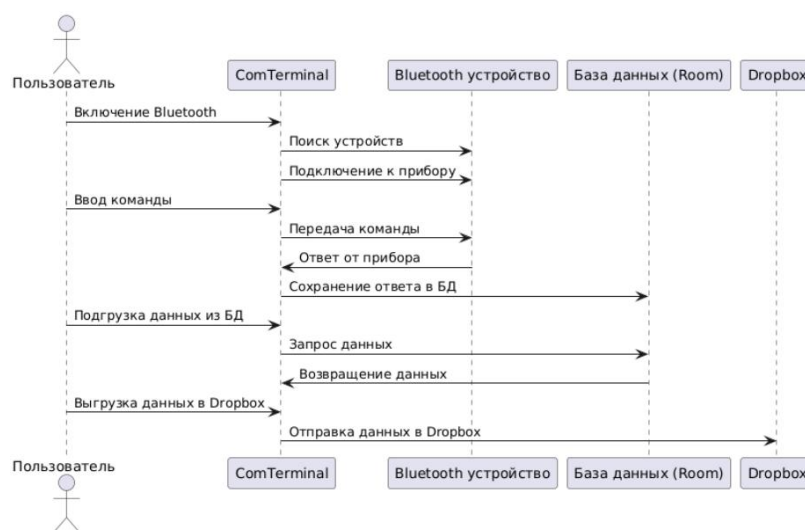


Рисунок 8 – Взаимодействие пользователя с мобильным приложением

4.2.2 *Определение операций пользователей и составление функциональных блоков*

Проанализировав предметную область и выделив профили потенциальных пользователей, можно определить операции пользователей, которые они могут выполнять в рамках функций предложенного им приложения:

- 1) Включение Bluetooth на устройстве
- 2) Разрешение на доступ к хранилищу устройства
- 3) Разрешение на доступ к геопозиции
- 4) Поиск доступных Bluetooth-устройств
- 5) Выбор новых Bluetooth-устройств
- 6) Выбор уже запомненных Bluetooth-устройств
- 7) Подключение к выбранному Bluetooth-устройству
- 8) Обновление списка доступных Bluetooth-устройств
- 9) Разрыв соединения с подключенным Bluetooth-устройством
- 10) Ввод команды для отправки на устройство
- 11) Отправка команды на устройство
- 12) Отображение полученного ответа на экране
- 13) Сохранение полученных данных в локальную базу данных
- 14) Загрузка сохраненных данных из локальной базы данных
- 15) Просмотр списка подключённых устройств (вкладка Devices)
- 16) Полная очистка подключённых устройств из БД
- 17) Просмотр сообщений терминала (вкладка Messages)
- 18) Выборочное удаление сообщения
- 19) Выборочная выгрузка данных в Dropbox
- 20) Массовое удаление сообщений
- 21) Просмотр журнала событий подключения/ошибок (вкладка Logs)
- 22) Выборочное удаление событий подключения/ошибок
- 23) Полная очистка логов событий
- 24) Просмотр истории выгрузок в Dropbox (вкладка Dropbox Uploads)
- 25) Очистка истории выгрузок

4.2.3 *Проектирование структуры экранов и схемы навигации*

Проделав анализ предметной области, сформировав профили и сценарии действий пользователей, определив функциональность приложения, можно составить схему навигационной системы, изображённую на рисунке 9.

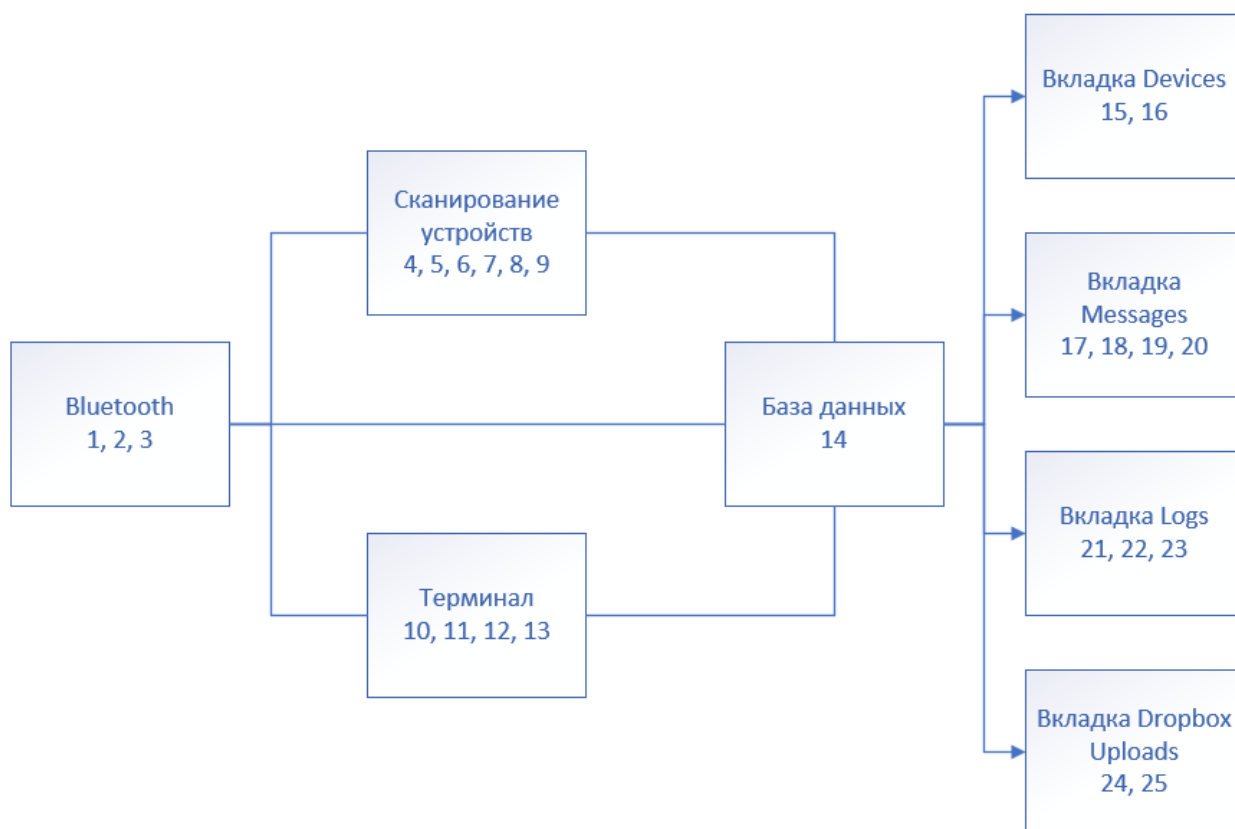


Рисунок 9 – схема навигации

4.2.4 Разработка дизайна интерфейса

Разработка интерфейса пользователя является важной частью процесса создания мобильного приложения, так как от неё напрямую зависит удобство взаимодействия пользователя с системой. На данном этапе определяются ключевые визуальные и навигационные решения, разрабатываются цветовая схема, структура экранов, стилистика элементов и поведение интерфейса при различных сценариях использования.

В рамках проектирования интерфейса была разработана схема навигации по основным экранам приложения ComTerminal. Центральным элементом схемы выступает стартовый экран, обеспечивающий доступ к четырём основным разделам приложения:

- Bluetooth — управление соединениями с внешними устройствами;
- Поиск устройств — отображение и подключение доступных Bluetooth-устройств;
- Терминал — взаимодействие с подключённым устройством путём приёма и отправки данных;
- База данных — доступ к сохранённой информации, включая сообщения, логи и историю загрузок.

Схема, представленная на рисунке 10, демонстрирует логическую структуру интерфейса, обеспечивая понимание архитектуры пользовательского взаимодействия. Все переходы между экранами организованы таким образом, чтобы пользователь мог легко и быстро перемещаться между разделами.



Рисунок 10 – Схема приложения

На следующем этапе были разработаны макеты экранных форм, отражающие композиционное и функциональное построение каждого интерфейса. Каждый макет создавался с учётом принципов удобства, минимализма и понятности.

Экран Bluetooth. Данный экран предоставляет информацию о текущем состоянии Bluetooth-модуля. Макет включает элемент управления модулем и переход к другим макетам приложения. Интерфейс разработан с акцентом на быстрое действие и простоту активации соединения. Визуальное представление макета экрана Bluetooth можно увидеть на рисунке 11.

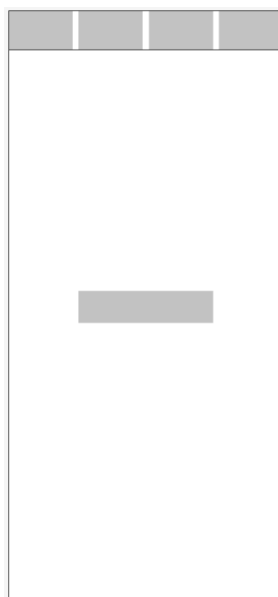


Рисунок 11 – Макет “Bluetooth”

Экран поиска устройств. На данном экране реализован механизм отображения найденных Bluetooth-устройств. В макете предусмотрены список доступных устройств с возможностью подключения, кнопки обновления и разрыва соединения. Визуальная структура упрощает выбор устройства за счёт читаемости и лаконичного отображения информации. Макет экрана поиска устройств приведён на рисунке 12.

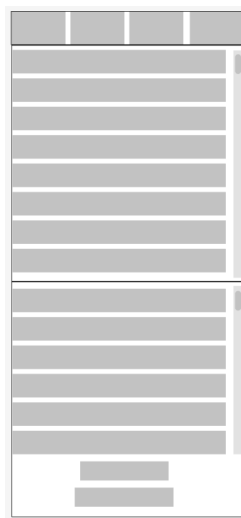


Рисунок 12 – Макет “Поиск устройств”

Экран терминала. Основной экран взаимодействия с подключённым устройством. Содержит поле отображения сообщений, строку ввода и кнопку отправки. Макет спроектирован таким образом, чтобы обеспечить максимальную функциональность при минимальных визуальных отвлечениях, а также предусмотреть возможность сохранения сообщений в базу данных. Изображение макета экрана терминала представлено на рисунке 13.

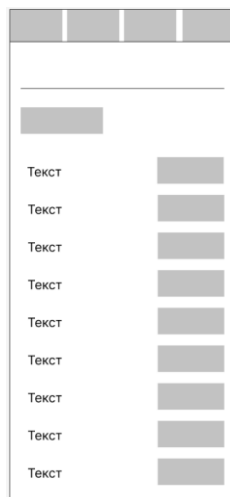


Рисунок 13 – Макет “Терминал”

Экран базы данных. Интерфейс предназначен для просмотра и управления сохранённой информацией. Структура экрана направлена на обеспечение быстрого доступа к информации с возможностью выполнения базовых операций. Соответствующий макет интерфейса базы данных отображён на рисунке 14.

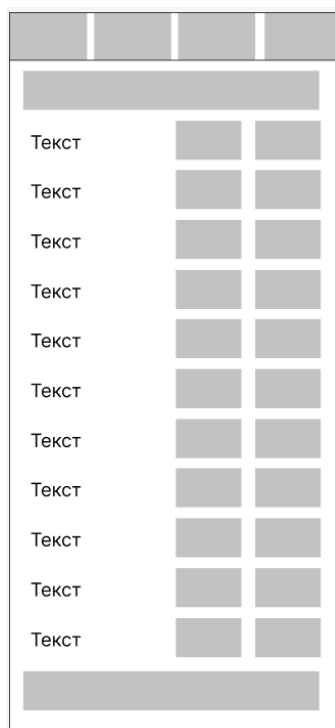


Рисунок 14 – Макет “База данных”

После завершения этапа макетирования были подготовлены графические шаблоны всех основных экранов приложения. Шаблоны представляют собой детализированные визуальные заготовки, в которых конкретизируются параметры оформления интерфейса: отступы, шрифты, размеры элементов, цветовые акценты и стилистика интерактивных компонентов. Разработка шаблонов позволила обеспечить единый визуальный стиль приложения и послужила основой для последующей реализации пользовательского интерфейса в коде.

Шаблон экрана Bluetooth. Шаблон уточняет расположение элементов управления, стилизацию кнопки, иконок и цветовые акценты. Пример графического шаблона экрана Bluetooth представлен на рисунке 15.

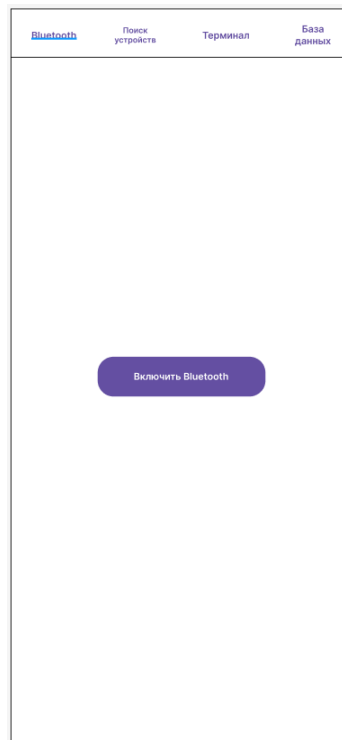


Рисунок 15 – Шаблон “Bluetooth”

Шаблон экрана Поиска устройств. Реализовано визуальное разграничение элементов списка, добавлены интерактивные состояния элементов (выделение, нажатие), реализована стилизация кнопок для соответствия общей теме интерфейса. Иллюстрация шаблона экрана поиска устройств приведена на рисунке 16.

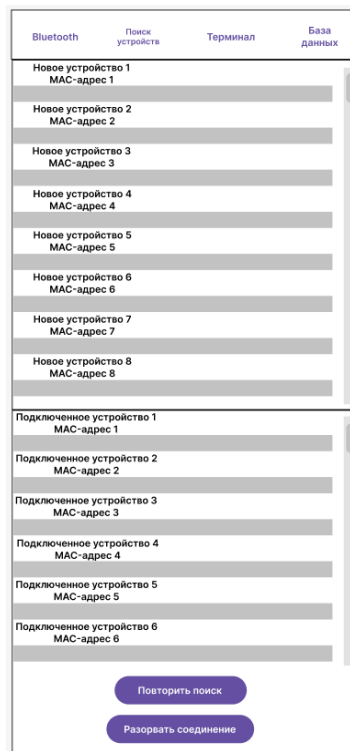


Рисунок 16 – Шаблон “Поиск устройств”

Шаблон экрана Терминала. Детализирована область отображения сообщений, заданы стили текстовых элементов и кнопок управления. Интерфейс адаптирован для работы в реальном времени. Графическое оформление шаблона терминала отображено на рисунке 17.

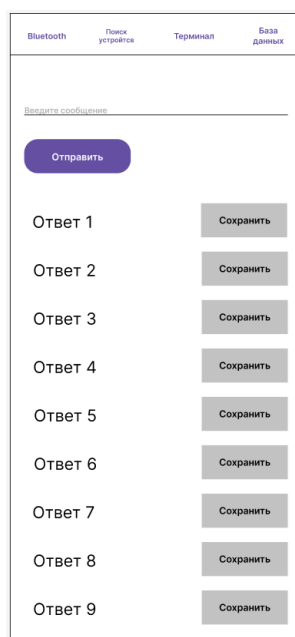


Рисунок 17 – Шаблон “Терминал”

Шаблон экрана Базы данных. Отображение реализовано в виде вкладок с таблицами, каждая из которых оформлена в едином стиле. Обеспечено единообразие элементов, реализованы интерактивные кнопки управления, адаптированные под мобильный формат. Шаблон интерфейса базы данных можно увидеть на рисунке 18.



Рисунок 18 – Шаблон “База данных”

4.3 Входные и выходные данные

Для эффективного функционирования мобильного приложения ComTerminal и обеспечения взаимодействия пользователя с устройством через Bluetooth необходимы входные и выходные данные.

Входные данные представляют собой информацию, которую пользователь вводит в приложение вручную. Это может включать текстовые данные или отдельные спецсимволы. Входные данные используются для управления устройствами, отправки команд и обработки полученных данных.

Входными данными для приложения являются:

- Команды управления устройством.

Выходные данные представляют собой информацию, полученную от подключённого устройства и отображаемую в приложении.

Выходные данные в приложении ComTerminal являются:

- Сообщения, полученные от устройства через Bluetooth.
- Ошибки передачи данных или сбой соединения.
- Уведомления о состоянии соединения Bluetooth.
- Сохранённые сообщения в базе данных
- Уведомления о удалении данных из базы данных.
- Подтверждения успешной или неудачной отправки данных в Dropbox.

4.4 Реализация используемых методов хранения, обработки и передачи информации об объектах предметной области

4.4.1 Методы хранения данных

База данных приложения ComTerminal построена на реляционной модели и включает в себя основную таблицу для хранения сообщений.

На основании логического проектирования данных была создана база данных, содержащая несколько таблиц, предназначенных для хранения и обработки сообщений, полученных через Bluetooth (см.таблицы 15-18).

Таблица 15 – Описание полей таблицы “Устройства” (device)

Имя поля	Тип данных	Свойства поля	Тип поля
id	INT	Длинное целое, автоинкремент	Ключевое
name	TEXT	Строка, неограниченная длина	Обязательное

timestamp	LONG	Длинное целое	Обязательное
address	TEXT	Строка, неограниченная длина	Обязательное

Таблица 16 – Описание полей таблицы “Сообщений” (terminal_messages)

Имя поля	Тип данных	Свойства поля	Тип поля
id	INT	Длинное целое, автоинкремент	Ключевое
message	STRING	Длина до 255 символов	Обязательное
timestamp	LONG	Длинное целое	Обязательное

Таблица 17 – Описание полей таблицы “Журнал логов” (logs)

Имя поля	Тип данных	Свойства поля	Тип поля
id	INT	Длинное целое, автоинкремент	Ключевое
device_id	INT	Длинное целое	Внешний ключ
timestamp	LONG	Длинное целое	Обязательное
status	TEXT	Строка, неограниченная длина	Обязательное

Таблица 18 – Описание полей таблицы “Загрузки в Dropbox” (dropbox_uploads)

Имя поля	Тип данных	Свойства поля	Тип поля
id	INT	Длинное целое, автоинкремент	Ключевое
message_id	INT	Длинное целое	Внешний ключ
timestamp	LONG	Длинное целое	Обязательное
status	TEXT	Строка, неограниченная длина	Обязательное

Наглядное представление структуры базы данных, показывающее отношения между таблицами, полями и другими объектами представлено на рисунке 19.

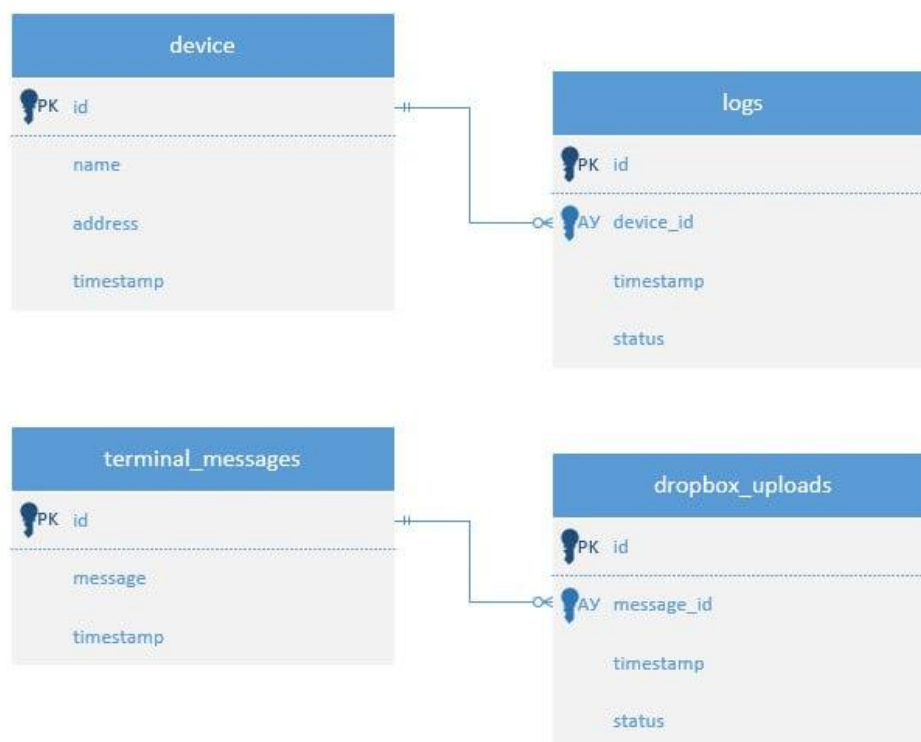


Рисунок 19 – Схема базы данных

Характеристика связей между таблицами:

В базе данных, представленной на рисунке 19, реализованы следующие связи между таблицами:

device — logs. Связь один-ко-многим: одно устройство может иметь несколько логов. Таблица logs содержит внешний ключ device_id.

terminal_messages — dropbox_uploads. Связь один-ко-многим: одно сообщение может быть загружено в Dropbox несколько раз. Таблица dropbox_uploads содержит внешний ключ message_id.

4.4.5 Алгоритмы использования технологий передачи данных

В процессе разработки программной системы ComTerminal были реализованы алгоритмы передачи данных как между внутренними компонентами приложения, так и между системой и внешними источниками данных — в частности, Bluetooth-устройствами и облачным хранилищем.

4.4.5.1 Межмодульная передача данных

Передача данных между модулями Android-приложения осуществляется с использованием общих объектов, передаваемых через основную активность

(MainActivity). Например, объект BluetoothSocket, установленный в FragmentDeviceScan, далее используется в FragmentTerminal для организации потока ввода/вывода информации. Такой подход обеспечивает единое соединение и согласованную передачу данных между модулями.

Для хранения и обмена структурированной информацией между компонентами приложения используется общая локальная база данных SQLite, управляемая с помощью библиотеки Room. Она выполняет роль централизованного хранилища и точки синхронизации данных. Доступ к базе осуществляется через DAO-интерфейсы, обеспечивающие:

- Выборку данных (@Query).
- Добавление (@Insert).
- Удаление (@Delete).
- Обновление (@Update).

Обращения к базе данных выполняются асинхронно, исключая блокировку UI-потока.

Структура БД отражает ключевые объекты системы:

- devices — подключённые устройства.
- terminal_messages — полученные/отправленные сообщения.
- logs — логирование событий.
- dropbox_uploads — история выгрузок в облако.

Таким образом, база данных играет роль посредника между модулями и обеспечивает согласованную передачу и хранение информации.

4.4.5.2 Передача данных с внешними источниками

Для связи с внешними устройствами применяется протокол Bluetooth RFCOMM, реализующий сериализацию данных по аналогии с COM-портом. Обмен данными организован следующим образом:

- Сканирование устройств — через BluetoothAdapter.startDiscovery().
- Установка соединения происходит через отдельную функцию BluetoothDevice.createRfcommSocketToServiceRecord(UUID).
- Передача данных — с использованием OutputStream и InputStream.

Адресация Bluetooth-устройств осуществляется по MAC-адресам, присваиваемым устройствам на уровне аппаратного обеспечения.

Обмен данными реализован в символьной форме (по строкам). При этом приём данных выполняется в отдельном потоке, который непрерывно слушает входной поток сокета. Полученные строки передаются в UI и/или сохраняются в базу данных.

Пример алгоритма приема данных:

- 1) Получить InputStream из BluetoothSocket;
- 2) Читать поток посимвольно до символа \n;
- 3) Собранный строку передать в UI и Room.

Также реализована система выгрузки сообщений в облачное хранилище Dropbox. Передача будет происходить по протоколу HTTP с использованием REST API Dropbox.

Алгоритм передачи включает следующие шаги:

- 1) Выбор сообщения пользователем (по кнопке "Отправить в Dropbox")
- 2) Формирование JSON-структуры из содержимого сообщения
- 3) Аутентификация с использованием токена доступа Dropbox
- 4) Отправка данных в фоновом потоке
- 5) Запись результата в таблицу dropbox_uploads (успешно/ошибка, дата)

Таблица 19 демонстрирует используемые технологии и протоколы передачи данных на разных уровнях взаимодействия в программной системе.

Таблица 19 – Передача данных в системе

Уровень	Средство передачи данных	Направление	Протокол/технология
Межмодульный	BluetoothSocket, Room (DAO)	Фрагмент – база данных	Java-код, Android SDK
Приложение – устройство	InputStream / OutputStream	Двухстороннее	Bluetooth RFCOMM (SPP)
Приложение – облако(Dropbox)	HTTP + REST API	Одностороннее (выгрузка)	HTTPS, OAuth2

4.5 Описание архитектурного решения

4.5.1 Структурная организация программной системы

Структура организации программной системы ComTerminal построена по принципу разделения ответственности между компонентами приложения с учетом специфики мобильной платформы Android. Приложение реализовано на языке Java и использует архитектурные компоненты Android Jetpack, включая Room, ViewModel и

Fragment. Также применяется библиотека Bluetooth API для взаимодействия с внешними устройствами и встроенные средства Android для управления ресурсами.

Компоненты системы:

1. Пользовательский интерфейс (UI):

- Реализован с использованием Activity, Fragment, ViewPager2 и TabLayout.
- Визуализирует данные, хранящиеся в локальной базе данных, и отображает информацию от Bluetooth-устройств в реальном времени.

- Обеспечивает навигацию между основными разделами: устройства, сообщения, логи и история загрузок в Dropbox.

2. Слой бизнес-логики (ViewModel):

- Обеспечивает взаимодействие между пользовательским интерфейсом и слоем данных.

- Управляет состоянием интерфейса, выполняет обработку данных перед отображением.

- Реализует обработку событий от пользователя (например, подключение/отключение Bluetooth, отправка в Dropbox).

3. Слой данных:

- Используется библиотека Room для работы с локальной базой данных SQLite.
- Реализованы сущности (@Entity) и DAO-интерфейсы (@Dao) для таблиц:
- Devices — подключённые устройства.
- Messages — сообщения, полученные от устройств.
- Logs — события соединения, отключения и ошибки.
- DropboxUploads — история загрузок сообщений в облако.
- Поддерживается фильтрация, добавление, удаление и обновление данных.

4. Bluetooth-модуль:

- Использует BluetoothAdapter, BluetoothDevice, BluetoothSocket и InputStream/OutputStream.

- Обеспечивает подключение к внешним устройствам, обмен сообщениями, обработку ошибок соединения.

- Интегрирован с системой логирования и базой данных.

5. Dropbox-интеграция:

- Осуществляется через Dropbox API.
- Позволяет выгружать выбранные сообщения в облачное хранилище.
- Каждое успешное действие записывается в таблицу DropboxUploads.

Преимущества такой модификации:

- Модульность: Разделение приложения на логически обособленные слои упрощает разработку, тестирование и сопровождение.
- Локальное хранение: Использование Room обеспечивает надёжное и производительное локальное хранилище с возможностью запросов и реактивного обновления UI.
- Реальное время: Работа с Bluetooth происходит асинхронно и позволяет обрабатывать данные в реальном времени.
- Устойчивость к сбоям: Логирование и сохранение всех данных позволяют восстановить историю соединений и сообщений даже после перезапуска приложения.
- Масштабируемость: Архитектура позволяет легко добавлять новые источники данных, типы подключений или способы экспорта информации.

Все фрагменты приложения взаимодействуют с соответствующими сущностями, такими как подключённые устройства, сообщения, логи и история загрузок в Dropbox. Архитектурные взаимосвязи между основными компонентами системы и их размещение на физическом устройстве наглядно отражены на диаграммах компонентов и развертывания UML, представленных на рисунках 20 и 21 соответственно.

Компонентная диаграмма - ComTerminal

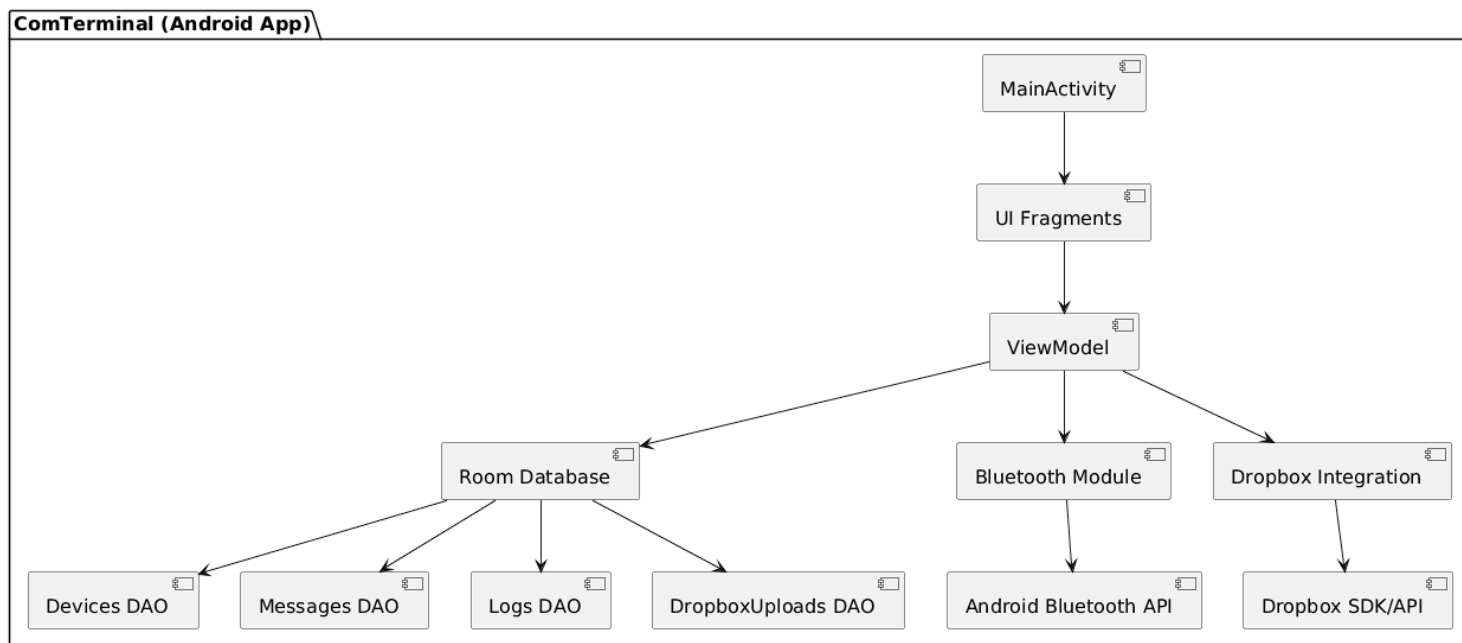


Рисунок 20 – Диаграмма компонентов

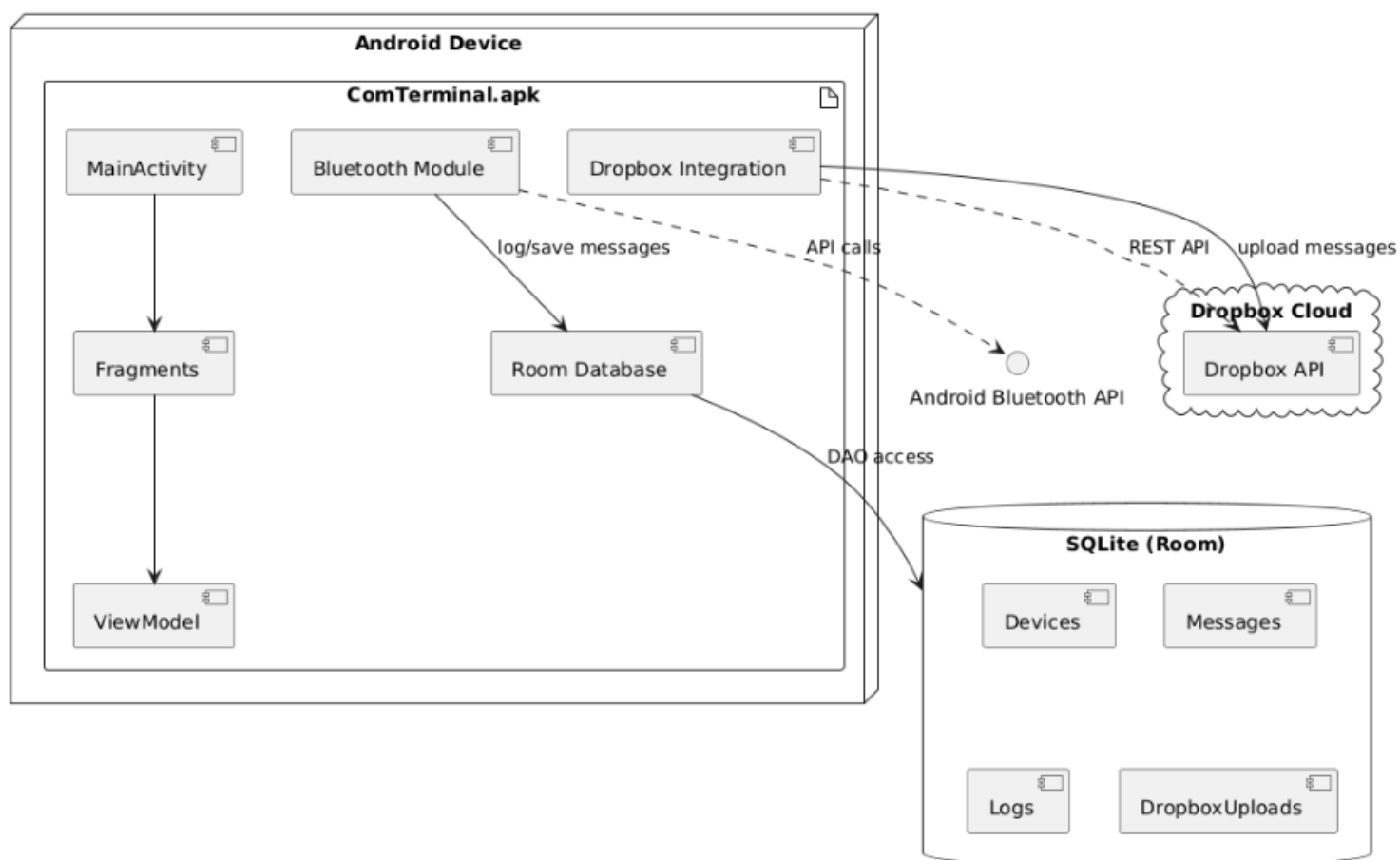


Рисунок 21 – Диаграмма развёртывания

4.5.2 Архитектура программного кода

Программная система ComTerminal реализована на языке Java в среде Android Studio. Архитектура кода построена по модульному принципу, где каждый класс отвечает за определённую задачу: работу с Bluetooth, передачу данных, отображение, хранение сообщений и логов. Код не следует строго шаблонам MVC или MVVM, однако содержит логическое разделение по функциональным блокам. Для лучшей наглядности структура системы представлена в виде таблиц:

Таблица 20 - Фрагменты

Фрагмент	Назначение	Основные методы
FragmentBluetooth	Управление состоянием Bluetooth-адаптера	onCreateView(), toggleBluetooth()
FragmentScanDevices	Сканирование и подключение к Bluetooth-устройствам	checkAndRequestPermissions(), startBluetoothScan(), connectToDevice(), saveDeviceToDatabase(), saveLogToDatabase(), saveDisconnectionLog()

Фрагмент	Назначение	Основные методы
FragmentTerminal	Обмен данными через Bluetooth	sendMessage(String data), startListeningForData(), saveMessage(String data), saveDisconnectionLog()
FragmentDatabaseView	Интерфейс работы с локальной базой данных	onViewCreated()

Таблица 21 – Вкладки во фрагменте базы данных

Фрагмент (вкладка)	Назначение	Основные методы
DevicesFragment	Управление устройствами	loadData(), clearDatabase()
MessagesFragment	Работа с сообщениями	loadData(), clearDatabase(), clearMessages()
LogsFragment	Работа с логами устройств	loadData(), clearMessages(), clearLogs()
DropboxUploadsFragment	Управление загрузками в Dropbox	loadUploads(), clearLogs(), clearUploads()

Таблица 22 – Адаптер

Класс	Назначение	Основные методы
DatabasePagerAdapter	Управление фрагментами базы данных	saveMessage(String data), createFragment(int position), getItemCount()

Таблица 23 – Модельные классы базы данных

Модель	Назначение	Поля
TerminalMessage	Хранение сообщений терминала	id: int, message: String, timestamp: long
Device	Хранение информации об устройствах	id: int, name: String, address: String, timestamp: long
LogEntry	Хранение логов подключения/отключения	id: int, device_id: int, timestamp: long, status: String
DropboxUpload	Хранение истории загрузок в Dropbox	id: int, message_id: int, timestamp: long, status: String

Связи

- Bluetooth инициирует сканирование через DeviceScan.
- DeviceScan обнаруживает устройства и сохраняет их в базу данных.

- DeviceScan сохраняет логи подключения/отключения через LogEntry.
- DeviceScan передает данные в Terminal.
- Terminal сохраняет сообщения через TerminalMessage.
- FragmentDatabaseView использует DatabasePagerAdapter для работы с фрагментами.
- DevicesFragment, MessagesFragment, LogsFragment, и DropboxUploadsFragment содержат свои соответствующие фрагменты.

Все фрагменты взаимодействуют с соответствующими сущностями, такими как устройства, сообщения, логи и загрузки, что наглядно отражено в диаграмме классов UML на рисунке 22.

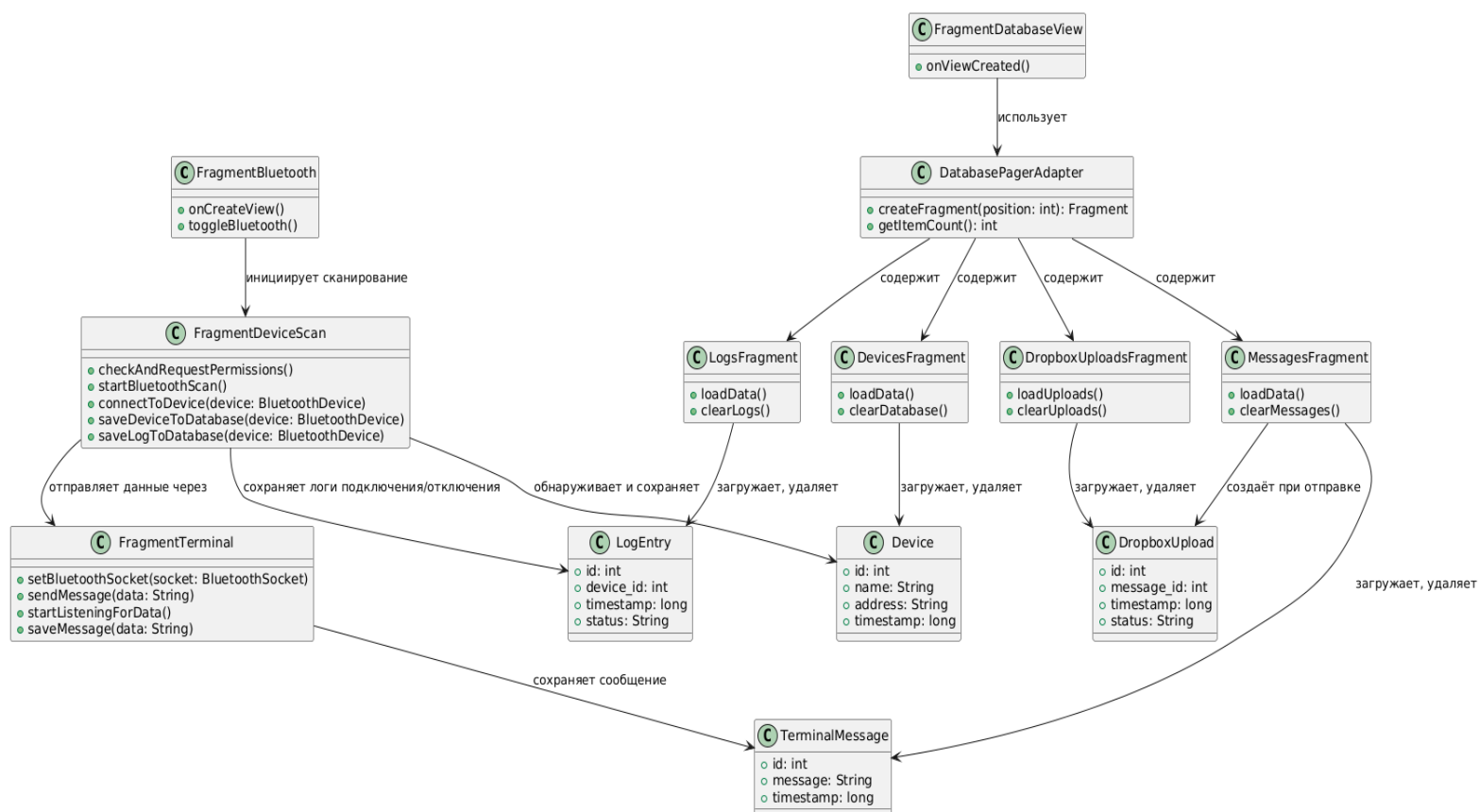


Рисунок 22 – Диаграмма классов UML мобильного приложения ComTerminal

5 ТЕСТИРОВАНИЕ И ОПТИМИЗАЦИЯ

5.1 План тестирования

Одним из ключевых этапов разработки программного продукта является его тестирование. Пропуск или недостаточное внимание к этому этапу нередко приводит к ошибкам, выявляемым уже после релиза, что может повлечь за собой негативные отзывы пользователей, снижение доверия к продукту и увеличение затрат на исправление недочётов. Особенно это актуально для программных средств, работающих с внешними устройствами и зависящих от нестабильных факторов среды, как, например, ComTerminal, взаимодействующий с физическим прибором по Bluetooth.

Bluetooth-соединения подвержены ряду факторов, которые крайне сложно или невозможно полноценно эмулировать в условиях автоматического тестирования. Это нестабильность сигнала, неожиданные обрывы связи, реакции реального прибора на команды, влияние аппаратных ограничений конкретного устройства. В связи с этим, основной упор при тестировании приложения делается на ручную проверку сценариев взаимодействия с прибором.

Ручное тестирование позволяет в полной мере оценить поведение приложения в условиях, приближённых к реальной эксплуатации. Тестировщик может имитировать поведение пользователя, проверять реакции системы на нестандартные ситуации, отслеживать корректность отображения информации в интерфейсе, оценивать стабильность Bluetooth-соединения и точность обработки данных, поступающих от прибора.

Тем не менее, при наличии повторяющейся логики, не связанной напрямую с оборудованием, возможно применение автоматического модульного тестирования. Например, для проверки корректности сохранения сообщений в базу данных, работы фильтрации, сериализации данных или валидации параметров API Dropbox.

Таким образом, основной акцент тестирования ComTerminal делается на ручную проверку сценариев, так как она позволяет выявить критические ошибки, возникающие при взаимодействии с физическим прибором. Автоматические тесты применяются выборочно, для локальных участков логики, не требующих аппаратного окружения.

5.2 Результаты тестирования и оптимизация

При разработке программного средства ComTerminal использовалось внешнее устройство — лазерный дальномер, оснащённый встроенным датчиком температуры. Дальномер способен передавать измеренные значения расстояния, температуры по Bluetooth-соединению с помощью команд. На рисунке 23 представлено данное устройство.

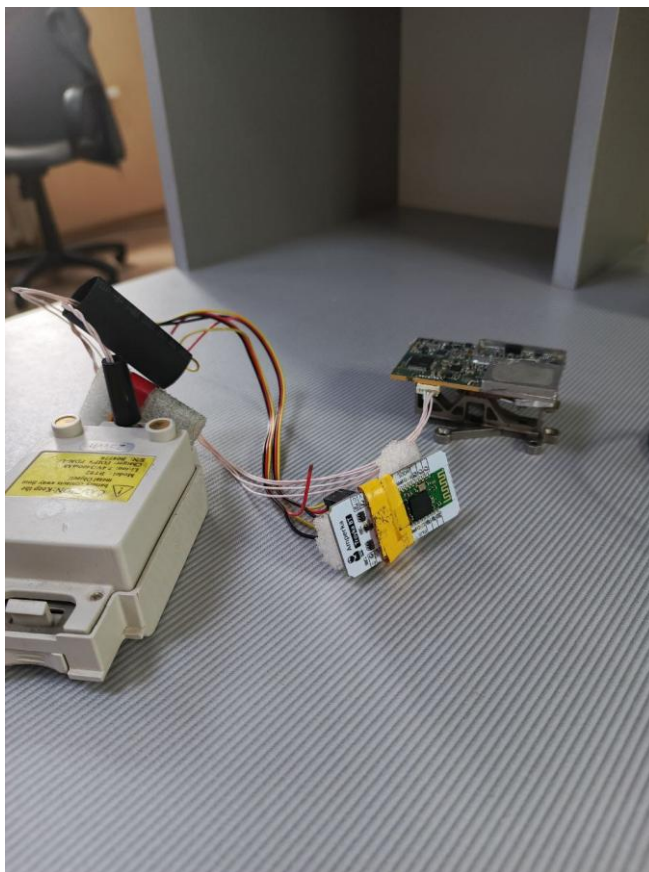


Рисунок 23 – Общее представление дальномера

Связь между мобильным приложением и прибором осуществляется с помощью Bluetooth-модуля HC-05 Troyka, который поддерживает протокол UART. Этот модуль используется для беспроводной передачи данных между микроконтроллерами и мобильными устройствами. На рисунке 24 изображён внешний вид модуля HC-05.

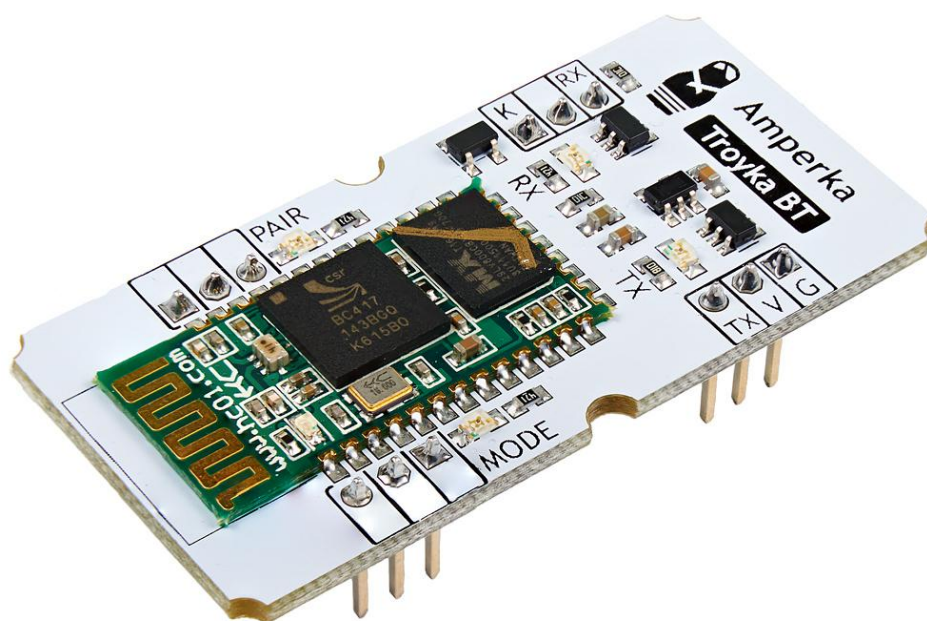


Рисунок 24 – Bluetooth-модуль HC-05 Troyka

Модуль оснащён несколькими индикаторами состояния, которые отображают текущие процессы. На рисунке 25 показаны элементы модуля, включая светодиоды, отвечающие за разные функции. Каждый из индикаторов имеет своё значение: PAIR — отображает состояние сопряжения, MODE — показывает режим работы, TX и RX — информируют о передаче и приёме данных соответственно.

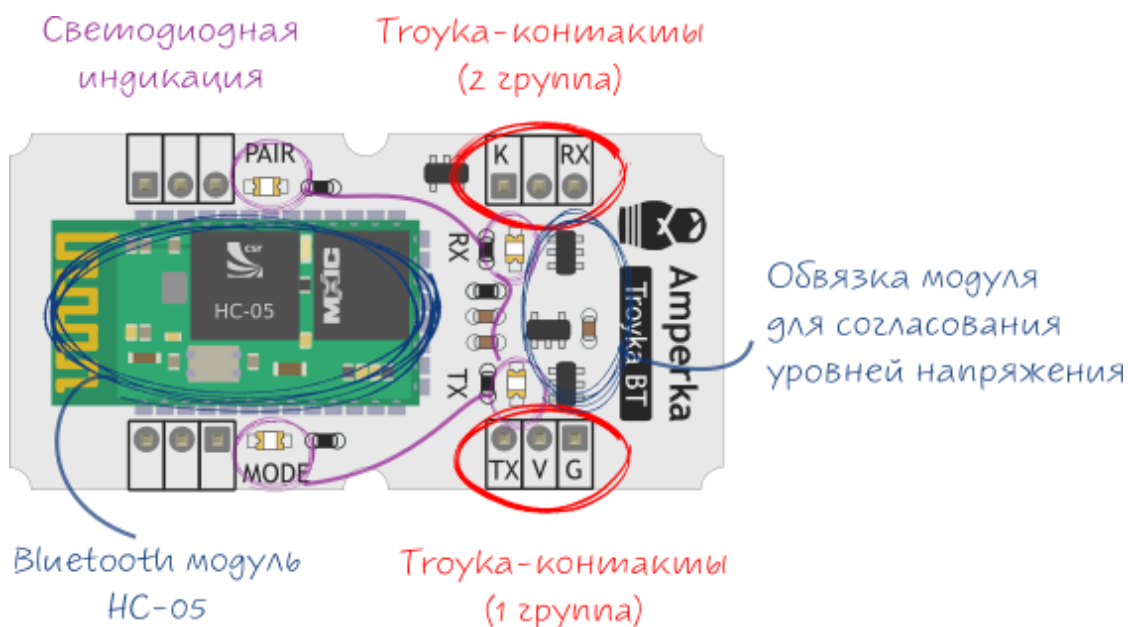


Рисунок 25 – элементы модуля HC-05

Первым этапом тестирования было установление Bluetooth-соединения между мобильным устройством и прибором. Для этого в приложении ComTerminal пользователь должен был выполнить несколько простых шагов:

- 1) Включение Bluetooth на мобильном устройстве;
- 2) Открытие интерфейса приложения для выбора доступных устройств;
- 3) Выбор нужного устройства из списка доступных (в данном случае это HC-05 Troyka);
- 4) Подключение к устройству, после чего индикатор PAIR на модуле HC-05 должен был начать гореть, подтверждая успешное сопряжение.

Этот этап тестирования проверял стабильность соединения, а также наличие визуального подтверждения через индикатор PAIR. На рисунке 26 показаны разделы Devices и Logs, где отображается информация о сопряжении телефона с прибором, а также подтверждается успешное подключение. На рисунке 27 изображены индикатор, подтверждающий успешное подключение модуля.

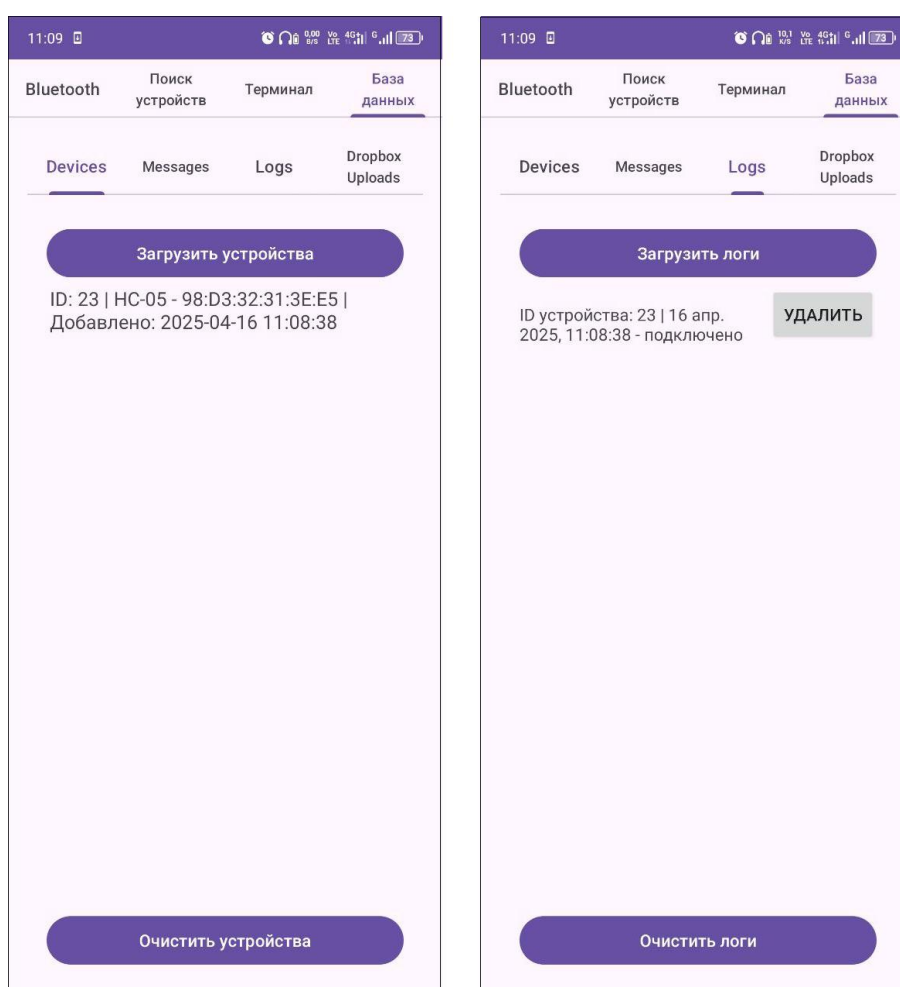


Рисунок 26 – Вкладки Devices и Logs с подключенным HC-05, с отображением успешного подключения устройства



Рисунок 27 – Индикация успешного подключения к прибору

После успешного подключения был протестирован процесс отправки команд от мобильного устройства на прибор. Для этого было введено несколько команд в текстовое поле терминала приложения, например:

- t - запрос температуры.

При отправке команды через интерфейс приложения, модуль HC-05 должен был передать запрос на прибор, который, в свою очередь, должен ответить с результатом значения температуры.

Индикаторы TX и RX на модуле HC-05 были использованы для контроля передачи данных: TX мигал при отправке данных, а RX — при их получении. На рисунке 28 изображён процесс ввода команды и отправки её на устройство.



Рисунок 28 – Терминал с ответом от прибора

После отправки команды и получения данных от устройства, приложение должно было отобразить ответ на экране. В нашем случае это было значение температуры, полученное от прибора.

Тестирование включало проверку корректности отображения данных в интерфейсе приложения. Полученные данные должны были отобразиться в соответствующем поле с правильным форматированием. На рисунке 29 показан пример отображения полученных данных в приложении после успешной отправки команды.



Рисунок 2 – Полученные данные от устройства

Для проверки стабильности работы соединения и корректности обработки нескольких команд, были проведены несколько быстрых последовательных тестов с одинаковым запросом. В каждом случае приложение должно было корректно отправить команду, получить данные, отобразить их и сохранить.

Индикаторы на модуле НС-05 продолжали мигать в зависимости от состояния передачи данных, что позволяло подтвердить, что данные успешно обрабатывались. На рисунке 30 показано стабильность работы приложения при отправках нескольких повторных команд и получение новых данных.

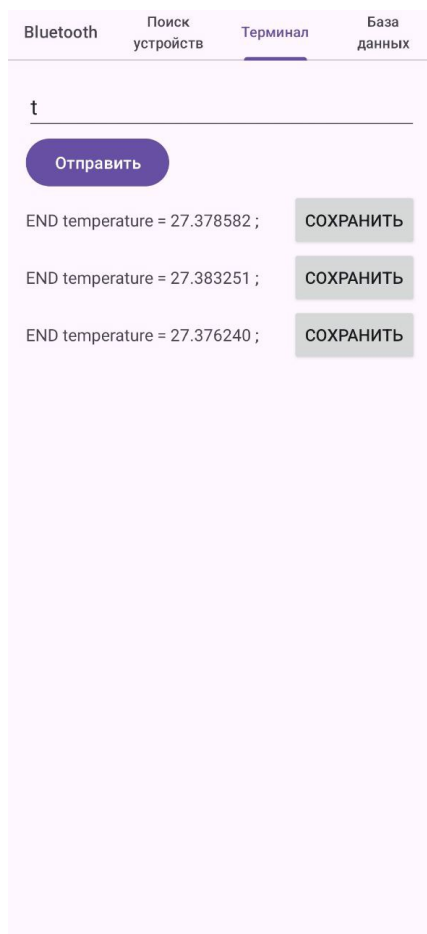


Рисунок 30 – Получение нескольких ответов от прибора

Далее проверялся функционал сохранения полученных данных в локальную базу данных приложения. После получения данных пользователь мог сохранить их для последующего использования. Для этого рядом с сообщением, отображённым в терминале, предусмотрена кнопка «Сохранить». По её нажатию данные заносились в таблицу Messages базы данных Room.

Дополнительно была протестирована возможность отправки сохранённых сообщений в облачное хранилище Dropbox. После сохранения пользователь мог воспользоваться кнопкой «Dropbox», расположенной рядом с каждым сообщением. При

успешной отправке соответствующая информация записывалась в таблицу Dropbox Uploads. На рисунке 31 показано, как данные сохраняются в локальную базу и отображаются после успешной выгрузки в облачное хранилище.

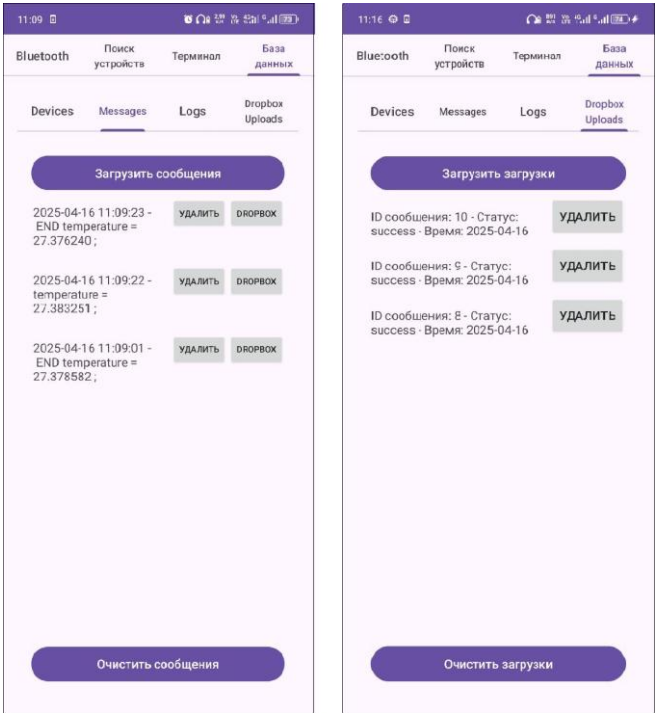


Рисунок 31 – Сохранение данных в локальной базе и выгрузка в облачное хранилище
Dropbox

Факт успешной выгрузки также подтверждается снимками экрана из интерфейса Dropbox, содержащими соответствующие данные на рисунке 32.

Dropbox / message_8 CSV			
Файл Правка Вид Справка			
	A	B	C
1	id	message	timestamp
2	8	END temperature = 27.378582 ;	1744777620117

Dropbox / message_9 CSV			
Файл Правка Вид Справка			
	A	B	C
1	id	message	timestamp
2	9	END temperature = 27.383251 ;	1744777724094

Dropbox / message_10 CSV			
Файл Правка Вид Справка			
	A	B	C
1	id	message	timestamp
2	10	END temperature = 27.376240 ;	1744777908199

Рисунок 32 – Таблицы message 8-10

Также был проведён тест по завершению сеанса связи. Для этого использовалась кнопка “Разорвать соединение” в интерфейсе приложения. После разрыва соединения в таблицу Logs автоматически добавлялась запись с указанием события и временем его возникновения. На рисунке 33 показан фрагмент таблицы Logs с зафиксированным фактом отключения устройства.

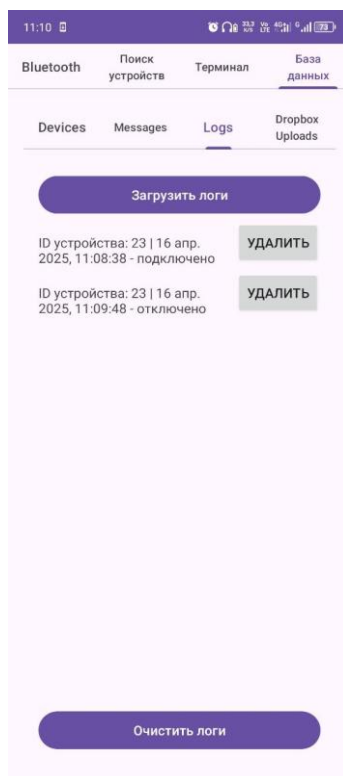


Рисунок 33 - Вкладка Logs с отображением успешного отключения устройства

6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Для того чтобы начать пользоваться мобильным приложением ComTerminal, необходимо сначала установить его на ваше Android-устройство. Скачивание приложения осуществляется следующим образом:

- 1) Получить APK-файл, сохранить его на устройстве;
- 2) Убедиться, что в настройках устройства разрешена установка приложений из неизвестных источников (Настройки → Безопасность → Неизвестные источники);
- 3) Открыть файл APK с помощью любого файлового менеджера и подтвердить установку;
- 4) После завершения установки можно открыть приложение ComTerminal с главного экрана или из списка приложений.

После установки и открытия приложения ComTerminal вы попадёте на главный экран приложения.

На рисунке 34 представлено страница начального экрана с кнопкой активации Bluetooth на устройстве.

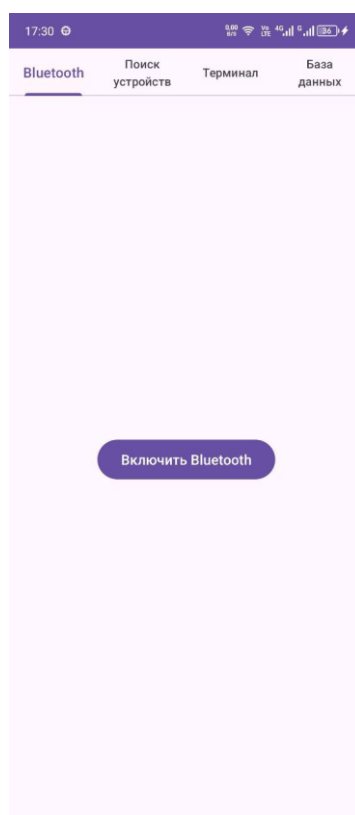


Рисунок 34 – Начальный экран ComTerminal

При первом запуске приложение запросит разрешение на использование Bluetooth. Нужно подтвердить разрешение, чтобы обеспечить корректную работу приложения. Показ разрешения реализован на рисунке 35.



Рисунок 35 – Разрешение на использование Bluetooth в приложении ComTerminal

Убедиться, что Bluetooth включён на вашем телефоне или планшете. Можно включить его через шторку уведомлений или, при необходимости, повторно нажать кнопку Bluetooth внутри приложения. На рисунках 36 показано, что Bluetooth успешно активирован.

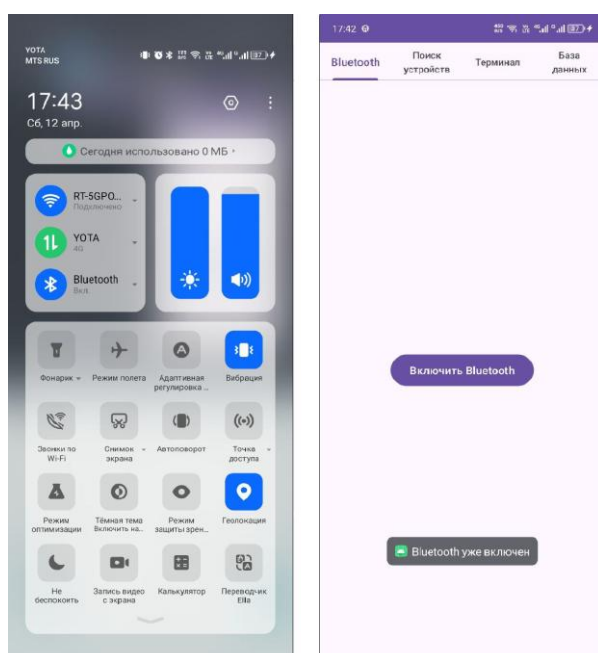


Рисунок 36 – Успешно активированный Bluetooth

Затем следует перейти на вкладку “Поиск устройств”. При запуске приложения в первый раз также потребует разрешение на доступ к геопозиции, это показано на рисунке 37.

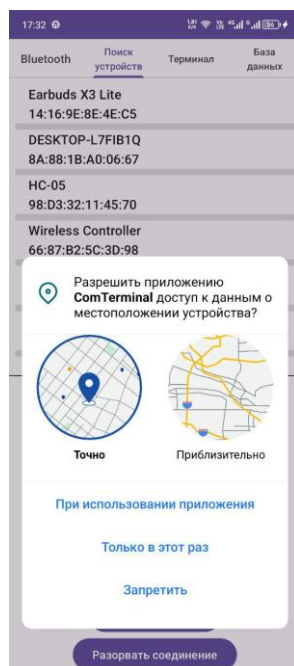


Рисунок 37 – Доступ к геопозиции приложения ComTerminal

После включения Bluetooth автоматически запустится поиск доступных устройств, также высветится уведомление. При необходимости можно выбрать ранее подключённое устройство из списка. Если устройство не было обнаружено с первого раза, то следует инициализировать поиск повторно, нажав соответствующую кнопку. Все действия показаны на рисунке 38.

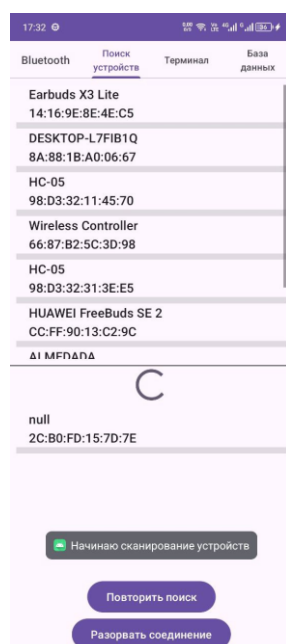


Рисунок 38 – Инициализация поиска новых устройств

После успешного подключения всплывёт уведомление на экране устройства.

Дополнительно можно перейти во вкладку Devices, чтобы просмотреть список всех устройств, с которыми ранее устанавливалось соединение.

А во вкладке Logs отображается история событий подключения: там можно увидеть, когда устройство было подключено или отключено. Пример успешного уведомления о подключении изображен на рисунках 39-41.

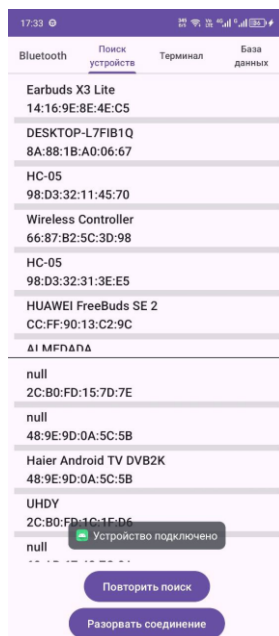


Рисунок 39 – Уведомление о успешном подключении к устройству



Рисунок 40 – Успешное подключение устройства в вкладке Device

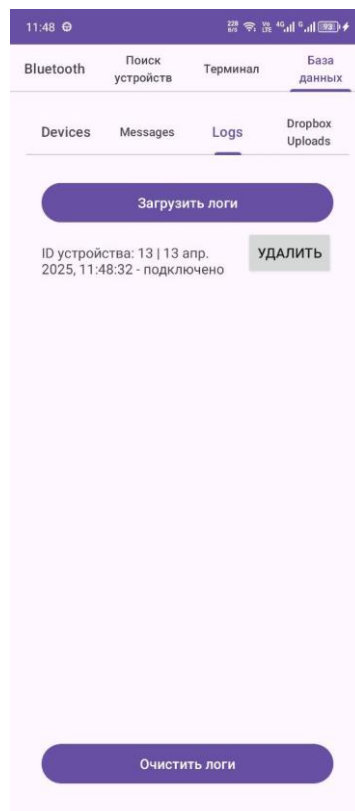


Рисунок 41 – Статус успешного подключения в вкладке Logs

В случае ошибки подключения приложение также отобразит уведомление — пример этого сообщения приведён на рисунке 42.

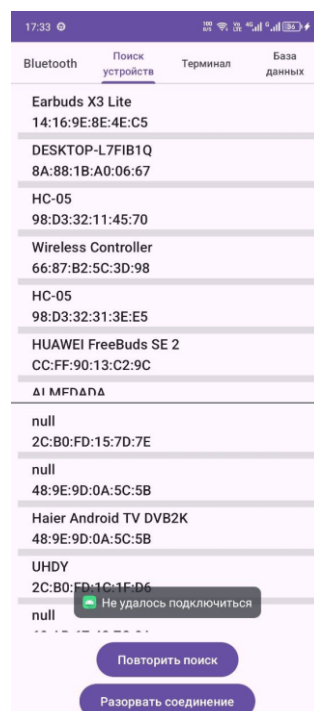


Рисунок 42 – Ошибка подключения к устройству

Если нужно разорвать соединение, следует перейти во вкладку "Поиск устройств" и нажать кнопку "Разорвать соединение". Также состояние подключения можно проверить во вкладке "Logs", как показано на рисунке 43.



Рисунок 43 – Статус отключения устройства во вкладке Logs

Если попытаться нажать кнопку "Разорвать соединение" повторно, появится уведомление о том, что в данный момент нет подключенного устройства. Это также продемонстрировано на рисунке 44.

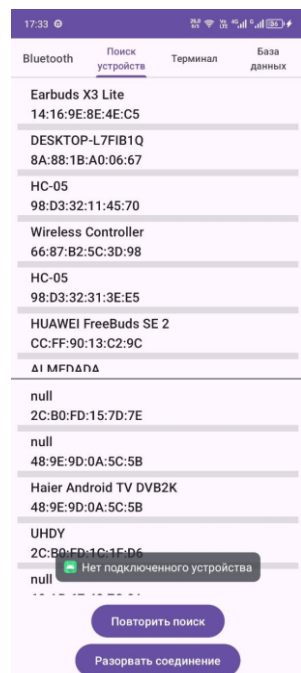


Рисунок 44 – Попытка повторного разрыва соединения с устройством

После успешного подключения к Bluetooth-устройству следует перейти на вкладку "Терминал", где можно отправлять сообщения и получать ответы.

Отправка сообщений:

- 1) Введите команду в поле ввода;
- 2) Нажмите кнопку “Отправить” для отправки сообщения на устройство;
- 3) Команда будет отправлена.

Получение ответа:

- 1) После отправки команды, терминал ответит на ваш запрос;
- 2) Ответ будет отображен в списке сообщений на той же вкладке.

После отправки команды в поле ввода, устройство возвращает ответ, который будет выведен в списке сообщений на той же вкладке терминала. Терминал с отображённым ответом устройства представлен на рисунке 45.

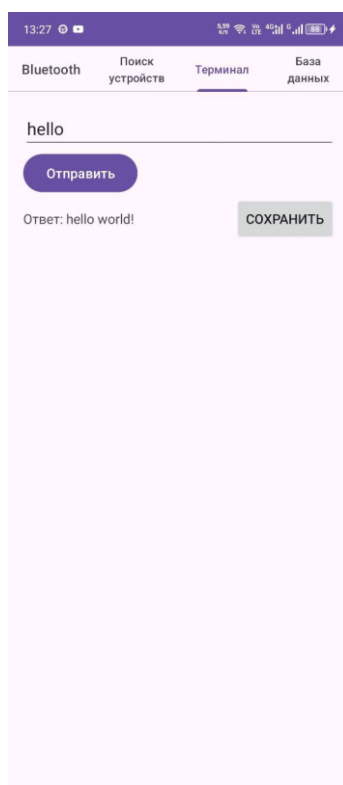


Рисунок 45 – Ответ устройства

Если необходимо сохранить полученные сообщения для последующего просмотра, это можно сделать с помощью кнопки “Сохранить”. Сохранённые данные автоматически попадают на вкладку Messages, где хранятся все полученные сообщения.

Для работы с сохранёнными данными в приложении предусмотрен раздел базы данных, который включает четыре вкладки: Devices, Messages, Logs и DropboxUploads. В каждой из вкладок можно загрузить данные из локальной базы и при необходимости выполнить полную очистку содержимого.

На вкладке Messages представлен список всех сообщений, сохранённых в локальной базе данных. Каждое сообщение содержит дату, текст и метку времени. Для удобства работы с данными здесь доступны функции удаления отдельного сообщения, а также выгрузки его содержимого в Dropbox.

Эта вкладка позволяет быстро находить, просматривать и экспортировать нужные сообщения. Пример интерфейса вкладки представлен на рисунке 46.

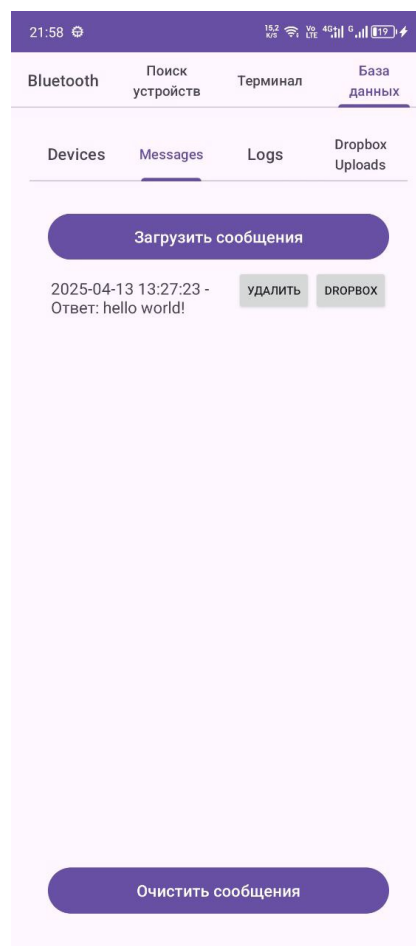


Рисунок 46 – Интерфейс “Messages”

Продолжая работу с сохранёнными сообщениями, пользователь может воспользоваться встроенной возможностью выгрузки отдельных записей в облачное хранилище Dropbox. Эта функция позволяет создать CSV-файл с содержимым выбранного сообщения и отправить его напрямую на сервер. Такой подход обеспечивает дополнительную надёжность хранения данных и возможность их дальнейшего использования вне приложения.

На рисунке 47 представлено уведомление об успешной отправке CSV-файла на сервер Dropbox.



Рисунок 47 – Загрузка CSV-файла на сервер Dropbox

После успешной отправки файла, система автоматически сохраняет информацию о выполненной операции в локальную базу данных. Вся история выгрузок становится доступной на отдельной вкладке “Dropbox Uploads”, где отображаются сведения о дате, статусе и идентификаторе выгруженного сообщения. Пример отображения вкладки приведён на рисунке 48.

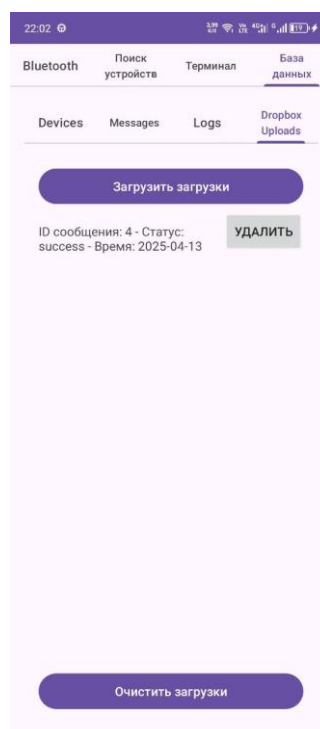


Рисунок 48 – Вкладка Dropbox Uploads

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы было разработано мобильное приложение ComTerminal, предназначенное для взаимодействия с внешними устройствами по протоколу Bluetooth. Разработка велась с учётом специфики работы с оборудованием и потребностей конечных пользователей, заинтересованных в удобной и стабильной передаче данных от приборов.

На первом этапе была проведена постановка задачи, включающая в себя определение ключевых функций приложения и требований к его архитектуре. Было выделено основное направление разработки — надёжное соединение с Bluetooth-устройствами, приём, отображение, хранение и выборочная отправка полученных сообщений.

Следующим этапом стало проектирование архитектуры приложения и локальной базы данных на основе Room, в которой реализовано хранение подключенных устройств, логов соединений, полученных сообщений и истории отправок в Dropbox.

В процессе реализации были внедрены ключевые функции:

- подключение и отключение Bluetooth-устройств;
- отправка команд и приём сообщений;
- сохранение данных в локальную базу;
- выборочная выгрузка данных в Dropbox с отслеживанием истории;
- логирование всех действий, связанных с соединением.

Важной частью работы стало тестирование, которое проводилось вручную, с подключением реального прибора.

Разработанное программное средство успешно выполняет поставленные задачи, обладает стабильной архитектурой и может быть использовано в реальных условиях для приёма данных от внешних приборов. Дальнейшее развитие проекта предполагает добавление поддержки дополнительных форматов данных, автоматическую синхронизацию с облачными сервисами и расширение интерфейса для работы с несколькими устройствами одновременно.

Таким образом, мобильное приложение ComTerminal полностью соответствует целям выпускной квалификационной работы и обладает высокой практической значимостью.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

- 1 Bluetooth SIG. Спецификация профиля Serial Port Profile (SPP). – [Электронный ресурс]. – URL <https://www.bluetooth.com/specifications/specs/serial-port-profile-1-2/> – (дата обращения: 16.04.2025).
- 2 Android Developers. Обзор Bluetooth. – [Электронный ресурс]. – URL <https://developer.android.com/guide/topics/connectivity/bluetooth/> – (дата обращения: 18.04.2025).
- 3 Google Codelabs. Работа с Room и LiveData. – [Электронный ресурс]. – URL <https://developer.android.com/codelabs/android-room-with-a-view> – (дата обращения: 19.04.2025).
- 4 Dropbox. Официальная документация API. – [Электронный ресурс]. – URL <https://www.dropbox.com/developers> (дата обращения: 19.04.2025).
- 5 Колисниченко Д. Н. Программирование для Android: учебник / Д. Н. Колисниченко. — Москва: Издательство Юрайт, 2023. — 350 с. — (Разработка программного обеспечения для мобильных устройств).
- 6 Metanit. Заголовки страниц и TabLayout. – [Электронный ресурс]. – URL: <https://metanit.com/java/android/9.2.php> – (дата обращения: 27.04.2025).
- 7 Фрайман З. Создание приложений для смартфонов и планшетов под ОС Android / З. Фрайман. — Москва: Издательство Вильямс, 2020. — 384 с. — (Разработка мобильных приложений).
- 8 Шабалин Ю. Permissions в Android: как не допустить ошибок при разработке / Ю.Шабалин. — [Электронный ресурс]. — URL: https://habr.com/ru/companies/swordfish_security/articles/741574/ – (дата обращения: 29.04.2025).

ПРИЛОЖЕНИЯ

Приложение А

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ (ВКР)

СТУДЕНТ: Чистяков Артём Александрович

ГРУППА: 107в1

СПЕЦИАЛЬНОСТЬ: 09.02.07 «Информационные системы и программирование»

ЗАКАЗЧИК: Ларичкин Алексей Юрьевич

НАЗВАНИЕ РАБОТЫ: Разработка Android-приложения с реализацией Serial Port Profile (SPP).

ОБЛАСТЬ ПРИМЕНЕНИЯ ПС: Сбор информации с датчиков в экспериментах на статическую прочность представляет собой комплексную задачу, связанную с проведением испытаний материалов, конструкций или узлов на их способность выдерживать статические нагрузки без разрушения или появления критических дефектов.

ОСНОВНЫЕ ПОЛЬЗОВАТЕЛЬСКИЕ ТРЕБОВАНИЯ:

Функционал:

- **Подключение через SPP:** возможность подключения Android-устройства к внешним устройствам через Bluetooth с использованием Serial Port Profile.
- **Отображение подключенных устройств:** список доступных Bluetooth-устройств, с указанием их имени и MAC-адреса. Возможность выбора устройства для подключения.
- **Мониторинг соединения:** отображение текущего состояния подключения (подключено/отключено). Обработка ошибок при попытке подключения или передаче данных.
- **Окно ввода/вывода текстовой информации:** реализация окна для ввода/вывода текстовой информации.

Внешний вид:

- **Интерфейс приложения:** простой и интуитивно понятный интерфейс. Наличие кнопок для: сканирования устройств (установления соединения), запуска Bluetooth.

Эксплуатационные характеристики:

- Высокая производительность при приеме, отображении и обработке сообщений от Bluetooth-устройств.

Требования к устройству:

- Устройство должно поддерживать Bluetooth и операционную систему Android версии 8.0 (API Level 26) или выше.

Языковая поддержка:

- Приложение на русском языке (опционально добавить английский).

УСЛОВИЯ РАЗРАБОТКИ ПС:

Программные средства:

- Язык программирования: Android Studio (основная разработка), Python (дополнительные модули, например, для обработки данных или тестирования).

Технологии и библиотеки:

- Bluetooth API для Android SDK. Java/Kotlin для основной разработки. Python (PySerial) для разработки вспомогательных скриптов или тестирования.

Оборудование: Смартфон на операционной системе Android (версии 8.0 (API Level 26) или выше с поддержкой Bluetooth), модуль Bluetooth, ПК, Прибор с поддержкой Bluetooth.

Сроки: до мая 2025 года, включая этапы проектирования, реализации и тестирования.

УСЛОВИЯ ТЕСТИРОВАНИЯ:

Данные для тестирования: Набор текстовых данных различного объема и содержания для проверки функционала.

Программно-аппаратное обеспечение: Тестирование будет проводиться на ПК с установленным Bluetooth модулем, а также с реальным Bluetooth-устройством.

Сроки тестирования: Тестирование будет проводиться в течение 2 недель после завершения разработки.

Подпись представителя:



/ Ларичкин А.Ю.

Приложение Б

FragmentBluetooth.java:

```
package com.example.comterminal;

import android.Manifest;
import android.bluetooth.BluetoothAdapter;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.os.Build;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.core.app.ActivityCompat;
import androidx.fragment.app.Fragment;

public class FragmentBluetooth extends Fragment {

    private BluetoothAdapter bluetoothAdapter; // Адаптер для работы с Bluetooth
    private Button bluetoothButton;          // Кнопка управления Bluetooth

    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState)
    {
        View view = inflater.inflate(R.layout.fragment_bluetooth, container, false);

        bluetoothAdapter = BluetoothAdapter.getDefaultAdapter(); // Получение адаптера
        bluetoothButton = view.findViewById(R.id.bluetoothButton); // Привязка кнопки

        // Проверка на поддержку Bluetooth
        if (bluetoothAdapter == null) {
            bluetoothButton.setText("Bluetooth not supported");
            bluetoothButton.setEnabled(false);
            return view;
        }

        // Обработчик нажатия кнопки
        bluetoothButton.setOnClickListener(v -> {
            // Проверка разрешения на Android 12+
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S &&
                ActivityCompat.checkSelfPermission(requireContext(),
Manifest.permission.BLUETOOTH_CONNECT) != PackageManager.PERMISSION_GRANTED) {
                requestPermissions(new String[]{Manifest.permission.BLUETOOTH_CONNECT}, 1);
            } else {
                toggleBluetooth();
            }
        });

        return view;
    }

    // Включение Bluetooth (если отключён)
    private void toggleBluetooth() {
```

```

        if (!bluetoothAdapter.isEnabled()) {
            startActivity(new Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE));
        } else {
            Toast.makeText(getApplicationContext(), "Bluetooth уже включен", Toast.LENGTH_SHORT).show();
        }
    }
}

```

FragmentDatabaseView.java:

```
package com.example.comterminal;
```

```

import android.os.Bundle;
import android.view.View;
import androidx.fragment.app.Fragment;
import androidx.viewpager2.widget.ViewPager2;

```

```

import com.example.comterminal.fragmentDatabase.DatabasePagerAdapter;
import com.google.android.material.tabs.TabLayout;
import com.google.android.material.tabs.TabLayoutMediator;

```

```
public class FragmentDatabaseView extends Fragment {
```

```

    public FragmentDatabaseView() {
        super(R.layout.fragment_database_view); // Привязка layout-файла к фрагменту
    }

```

```
@Override
```

```

    public void onViewCreated(View view, Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

```

```
        // Получаем ссылки на TabLayout и ViewPager2
```

```
        TabLayout tabLayout = view.findViewById(R.id.tabLayout);
```

```
        ViewPager2 viewPager = view.findViewById(R.id.viewPager);
```

```
        // Устанавливаем адаптер для ViewPager2
```

```
        DatabasePagerAdapter pagerAdapter = new DatabasePagerAdapter(this);
```

```
        viewPager.setAdapter(pagerAdapter);
```

```
        // Привязка табов к ViewPager2
```

```
        new TabLayoutMediator(tabLayout, viewPager, (tab, position) -> {
```

```
            switch (position) {
```

```
                case 0: tab.setText("Devices"); break;
```

```
                case 1: tab.setText("Messages"); break;
```

```
                case 2: tab.setText("Logs"); break;
```

```

        case 3: tab.setText("Dropbox Uploads"); break;
    }
    }).attach();
}
}

```

FragmentDeviceScan.java:

```

public class FragmentDeviceScan extends Fragment {

    private BluetoothAdapter bluetoothAdapter; // Адаптер Bluetooth — основная точка доступа к Bluetooth
    private ArrayAdapter<String> deviceAdapter; // Адаптер для отображения найденных устройств
    private ArrayList<String> deviceList; // Список найденных устройств
    private BluetoothSocket socket; // Сокет активного соединения с устройством

    private static final int REQUEST_BLUETOOTH_PERMISSIONS = 1;

    private ActivityResultLauncher<Intent> enableBluetoothLauncher; // Запрос на включение Bluetooth через
    Activity Result API
    private ProgressBar progressBar; // Прогресс-бар при сканировании

    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState)
    {
        View view = inflater.inflate(R.layout.fragment_device_scan, container, false);

        // Получаем Bluetooth-адаптер
        bluetoothAdapter = BluetoothAdapter.getDefaultAdapter();

        // Инициализируем список и адаптеры
        deviceList = new ArrayList<>();
        deviceAdapter = new ArrayAdapter<>(requireContext(), android.R.layout.simple_list_item_1, deviceList);

        ArrayList<String> connectedDevicesList = new ArrayList<>();
        ArrayAdapter<String> connectedDevicesAdapter = new ArrayAdapter<>(requireContext(),
        android.R.layout.simple_list_item_1, connectedDevicesList);

        ListView deviceListView = view.findViewById(R.id.deviceListView);
        ListView connectedDevicesListView = view.findViewById(R.id.connectedDevicesListView);
        deviceListView.setAdapter(deviceAdapter);
        connectedDevicesListView.setAdapter(connectedDevicesAdapter);

        progressBar = view.findViewById(R.id.progressBar);
    }
}

```

```

// Используется для обработки результата включения Bluetooth
enableBluetoothLauncher = registerForActivityResult(new ActivityResultContracts.StartActivityForResult(),
result -> {
    if (result.getResultCode() == android.app.Activity.RESULT_OK) {
        startBluetoothScan(); // если пользователь включил Bluetooth — начинаем сканирование
    } else {
        Toast.makeText(requireContext(), "Bluetooth не был включен", Toast.LENGTH_SHORT).show();
    }
});

// Загружаем уже готовые устройства
for (BluetoothDevice device : bluetoothAdapter.getBondedDevices()) {
    connectedDevicesList.add(device.getName() + "\n" + device.getAddress());
}
connectedDevicesAdapter.notifyDataSetChanged();

// Обработка клика по списку подключённых устройств
connectedDevicesListView.setOnItemClickListener((parent, view1, position, id) -> {
    String deviceInfo = connectedDevicesList.get(position);
    String deviceAddress = deviceInfo.substring(deviceInfo.indexOf("\n") + 1);
    BluetoothDevice device = bluetoothAdapter.getRemoteDevice(deviceAddress);
    connectToDevice(device); // иницилируем соединение
});

// Обработка клика по списку найденных устройств
deviceListView.setOnItemClickListener((parent, view1, position, id) -> {
    String deviceInfo = deviceList.get(position);
    String deviceAddress = deviceInfo.substring(deviceInfo.indexOf("\n") + 1);
    BluetoothDevice device = bluetoothAdapter.getRemoteDevice(deviceAddress);
    connectToDevice(device);
});

// Кнопка "Повторное сканирование"
Button repeatScanButton = view.findViewById(R.id.repeatScanButton);
repeatScanButton.setOnClickListener(v -> startBluetoothScan());

// Кнопка "Разорвать соединение"
Button sendMessageButton = view.findViewById(R.id.sendMessageButton);
sendMessageButton.setOnClickListener(v -> {
    if (socket != null && socket.isConnected()) {
        sendData(socket, "Соединение разорвано!");
    }
});

```

```

        saveDisconnectionLog(socket); // логируем отключение
    } else {
        Toast.makeText(requireContext(), "Нет подключенного устройства", Toast.LENGTH_SHORT).show();
    }
});

// Проверка разрешений Bluetooth
checkAndRequestPermissions();
}

// Проверка разрешений и их запрос при необходимости
private void checkAndRequestPermissions() {
    if (ContextCompat.checkSelfPermission(requireContext(), Manifest.permission.BLUETOOTH_SCAN) !=
PackageManager.PERMISSION_GRANTED
        || ContextCompat.checkSelfPermission(requireContext(),
Manifest.permission.BLUETOOTH_CONNECT) != PackageManager.PERMISSION_GRANTED
        || ContextCompat.checkSelfPermission(requireContext(),
Manifest.permission.ACCESS_FINE_LOCATION) != PackageManager.PERMISSION_GRANTED) {
        ActivityCompat.requestPermissions(requireActivity(),
            new String[]{
                Manifest.permission.BLUETOOTH_SCAN,
                Manifest.permission.BLUETOOTH_CONNECT,
                Manifest.permission.ACCESS_FINE_LOCATION
            },
            REQUEST_BLUETOOTH_PERMISSIONS);
    } else {
        startBluetoothScan(); // если всё уже разрешено — запускаем сканирование
    }
}

// Результат запроса разрешений
@Override
public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions, @NonNull int[]
grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
    if (requestCode == REQUEST_BLUETOOTH_PERMISSIONS) {
        if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
            startBluetoothScan(); // Сканируем
        } else {
            Toast.makeText(requireContext(), "Требуется разрешения Bluetooth",
Toast.LENGTH_SHORT).show();
        }
    }
}

```

```

    }
}

// Запуск сканирования устройств
private void startBluetoothScan() {
    if (ContextCompat.checkSelfPermission(requireContext(), Manifest.permission.BLUETOOTH_SCAN) ==
PackageManager.PERMISSION_GRANTED) {
        bluetoothAdapter.startDiscovery(); // начинаем поиск
        deviceList.clear(); // очищаем предыдущий список
        deviceAdapter.notifyDataSetChanged();
        progressBar.setVisibility(View.VISIBLE); // показываем прогресс
        Toast.makeText(requireContext(), "Начинаю сканирование устройств", Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(requireContext(), "Отсутствуют разрешения для сканирования Bluetooth",
Toast.LENGTH_SHORT).show();
    }
}

// Приёмник событий Bluetooth
private final BroadcastReceiver bluetoothReceiver = new BroadcastReceiver() {
// Установка соединения с выбранным устройством
private void connectToDevice(BluetoothDevice device) {
    if (socket != null && socket.isConnected()) {
        Toast.makeText(requireContext(), "Устройство уже подключено", Toast.LENGTH_SHORT).show();
        return;
    }

    new Thread() -> {
        try {
            // Используем стандартный UUID для Serial Port Profile (SPP)
            BluetoothSocket socket = device.createRfcommSocketToServiceRecord(UUID.fromString("00001101-
0000-1000-8000-00805F9B34FB"));
            bluetoothAdapter.cancelDiscovery(); // обязательно отменяем сканирование перед соединением
            socket.connect(); // соединяемся

// Сохраняем новое устройство в БД
private void saveDeviceToDatabase(BluetoothDevice device) {
    long currentTime = System.currentTimeMillis();
    Device newDevice = new Device(device.getName(), device.getAddress(), currentTime);

    AppDatabase db = AppDatabase.getInstance(requireContext());
    Executors.newSingleThreadExecutor().execute() -> db.deviceDao().insert(newDevice);
}

```

```

}

// Сохраняем лог подключения
private void saveLogToDatabase(BluetoothDevice device) {
    long currentTime = System.currentTimeMillis();
    AppDatabase db = AppDatabase.getInstance(requireContext());

    Executors.newSingleThreadExecutor().execute() -> {
        Device existingDevice = db.deviceDao().getDeviceByAddress(device.getAddress());
        if (existingDevice != null) {
            LogEntry logEntry = new LogEntry(existingDevice.id, currentTime, "подключено");
            db.logEntryDao().insert(logEntry);
        }
    });
}

// Отправка сообщения по Bluetooth-соединению
private void sendData(BluetoothSocket socket, String message) {
    new Thread() -> {
        try (OutputStream outputStream = socket.getOutputStream()) {
            outputStream.write(message.getBytes());
            outputStream.flush();
            Log.d("FragmentTerminal", "Сообщение отправлено: " + message);
        } catch (IOException e) {
            Log.e("FragmentTerminal", "Ошибка при отправке данных: " + e.getMessage());
        }
    }).start();
}
}

```

FragmentTerminal.java:

```

public class FragmentTerminal extends Fragment {

    // Сокет Bluetooth-соединения
    private BluetoothSocket bluetoothSocket;

    // Элементы интерфейса
    private TextView textViewOutput; // Поле для вывода сообщений
    private EditText editTextInput; // Поле для ввода сообщения
    private ScrollView mText_scroll_view;

    // Поток для общения с Bluetooth-устройством
    private OutputStream outputStream;
    private InputStream inputStream;
    private Handler handler = new Handler(Looper.getMainLooper());
    private AppDatabase database;

    // Метод, вызываемый при создании фрагмента
}

```



```

@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState) {
    // Загружаем layout
    View view = inflater.inflate(R.layout.fragment_terminal, container, false);

    // Получаем BluetoothSocket из MainActivity
    MainActivity activity = (MainActivity) requireActivity();
    BluetoothSocket socket = activity.getBluetoothSocket();

    // Устанавливаем Bluetooth-сокеты, если доступен
    if (socket != null) {
        setBluetoothSocket(socket);
    } else {
        Log.e("FragmentTerminal", "BluetoothSocket равен null. Проверьте соединение.");
    }

    // Получаем экземпляр базы данных
    database = AppDatabase.getInstance(requireContext());

    // Привязка UI-элементов
    textViewOutput = view.findViewById(R.id.textViewOutput);
    editTextInput = view.findViewById(R.id.editTextInput);
    Button buttonSend = view.findViewById(R.id.buttonSend);
    mText_scroll_view = view.findViewById(R.id.text_scroll_view);

    // Обработчик нажатия на кнопку "Отправить"
    buttonSend.setOnClickListener(v -> sendMessage());

    return view;
}

// Инициализация сокета и запуск приёма данных
public void setBluetoothSocket(BluetoothSocket socket) {
    this.bluetoothSocket = socket;
    try {
        outputStream = bluetoothSocket.getOutputStream(); // поток отправки
        inputStream = bluetoothSocket.getInputStream(); // поток приёма
        startListeningForData(); // слушаем входящие данные
    } catch (IOException e) {
        e.printStackTrace();
    }
}

// Отправка текста через Bluetooth
private void sendMessage() {
    String message = editTextInput.getText().toString(); // читаем текст из поля ввода
    if (!message.isEmpty()) {
        if (outputStream != null) {
            try {
                outputStream.write(message.getBytes()); // отправка данных
                outputStream.flush();
                mText_scroll_view.fullScroll(View.FOCUS_DOWN); // прокрутка вниз
            } catch (IOException e) {
                e.printStackTrace();
            }
        } else {
            Log.e("FragmentTerminal", "OutputStream равен null. Bluetooth-сокеты не инициализированы.");
        }
    }
}

```

```

}

// Поток приёма данных по Bluetooth
private void startListeningForData() {
    new Thread(() -> {
        byte[] buffer = new byte[1024]; // буфер для входящих данных
        int bytes;
        while (true) {
            try {
                bytes = inputStream.read(buffer); // читаем из потока
                String readMessage = new String(buffer, 0, bytes); // преобразуем в строку

                // Обновление UI в главном потоке
                handler.post(() -> {
                    textViewOutput.append(readMessage + "\n"); // отображаем
                    addMessageToLayout(readMessage); // добавляем с кнопкой
                });
            } catch (IOException e) {
                e.printStackTrace();
                break; // выход из цикла при ошибке
            }
        }
    }).start();
}

// Отображение нового сообщения с кнопкой "Сохранить"
private void addMessageToLayout(String messageText) {
    // Контейнер для сообщений (LinearLayout внутри ScrollView)
    LinearLayout containerLayout = mText_scroll_view.findViewById(R.id.messageContainer);

    // Горизонтальный контейнер: сообщение + кнопка
    LinearLayout messageLayout = new LinearLayout(getContext());
    messageLayout.setOrientation(LinearLayout.HORIZONTAL);
    messageLayout.setLayoutParams(new LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.MATCH_PARENT,
        LinearLayout.LayoutParams.WRAP_CONTENT));

    // Текст сообщения
    TextView messageView = new TextView(getContext());
    messageView.setText(messageText);
    messageView.setLayoutParams(new LinearLayout.LayoutParams(
        0,
        LinearLayout.LayoutParams.WRAP_CONTENT,
        1f)); // Вес 1: занимает всё доступное пространство

    // Кнопка "Сохранить"
    Button saveButton = new Button(getContext());
    saveButton.setText("Сохранить");
    saveButton.setLayoutParams(new LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.WRAP_CONTENT,
        LinearLayout.LayoutParams.WRAP_CONTENT));
    saveButton.setOnClickListener(v -> saveMessage(messageText)); // обработка клика

    // Добавляем элементы в горизонтальный layout
    messageLayout.addView(messageView);
    messageLayout.addView(saveButton);

    // Добавляем весь блок в контейнер
    containerLayout.addView(messageLayout);
}

```

```

    }

    // Сохранение сообщения в локальную БД (Room)
    private void saveMessage(String messageText) {
        new Thread() -> {
            TerminalMessage message = new TerminalMessage(messageText, System.currentTimeMillis());
            database.terminalMessageDao().insert(message); // вставка записи
        }.start();
    }
}

```

MainActivity.java:

```

public class MainActivity extends AppCompatActivity {

    private TabLayout tabLayout;
    private ViewPager2 viewPager;
    private BluetoothSocket bluetoothSocket;
    private FragmentTerminal terminalFragment;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        tabLayout = findViewById(R.id.tabLayout);
        viewPager = findViewById(R.id.viewPager);

        // Инициализация фрагментов
        terminalFragment = new FragmentTerminal();
        FragmentDatabaseView databaseFragment = new FragmentDatabaseView();

        // Устанавливаем адаптер для ViewPager2
        viewPager.setAdapter(new FragmentStateAdapter(this) {
            @NonNull
            @Override
            public Fragment createFragment(int position) {
                switch (position) {
                    case 0:
                        return new FragmentBluetooth();
                    case 1:
                        return new FragmentDeviceScan();
                    case 2:
                        // Если Bluetooth-сокет существует, передаем его в терминал
                        if (bluetoothSocket != null) {
                            terminalFragment.setBluetoothSocket(bluetoothSocket);
                        }
                        return terminalFragment;
                    case 3:
                        return databaseFragment;
                    default:
                        throw new IllegalArgumentException("Invalid position");
                }
            }
        });

        @Override
        public int getItemCount() {
            return 4; // Общее количество вкладок
        }
    }
}

```

```

// Связываем TabLayout с ViewPager2
new TabLayoutMediator(tabLayout, viewPager, (tab, position) -> {
    // Устанавливаем текст для каждой вкладки в зависимости от позиции
    switch (position) {
        case 0:
            tab.setText("Bluetooth");
            break;
        case 1:
            tab.setText("Поиск устройств");
            break;
        case 2:
            tab.setText("Терминал");
            break;
        case 3:
            tab.setText("База данных");
            break;
    }
}).attach(); // Подключаем TabLayout и ViewPager2
}

public BluetoothSocket getBluetoothSocket() {
    return bluetoothSocket;
}

public void setBluetoothSocket(BluetoothSocket socket) {
    this.bluetoothSocket = socket;
    // Если фрагмент терминала уже создан, обновляем его сокет
    if (terminalFragment != null) {
        terminalFragment.setBluetoothSocket(socket);
    }
}
}

```

AndroidManifest.xml:

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.comterminal">

    <!-- Разрешения для работы с Bluetooth -->
    <uses-permission android:name="android.permission.BLUETOOTH" />
    <uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
    <uses-permission android:name="android.permission.BLUETOOTH_SCAN"
android:usesPermissionFlags="neverForLocation" />
    <uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />

    <!-- Разрешение на интернет (если используется, например, Dropbox) -->
    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.ComTerminal">

        <activity
            android:name=".MainActivity"

```

```

        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

</application>
</manifest>

```

AppDatabase:

@Database(entities = {TerminalMessage.class, Device.class, LogEntry.class, DropboxUpload.class}, version = 4)
 public abstract class AppDatabase extends RoomDatabase {

```

        private static volatile AppDatabase INSTANCE;

        public abstract TerminalMessageDao terminalMessageDao();
        public abstract DeviceDao deviceDao();
        public abstract LogEntryDao logEntryDao();
        public abstract DropboxUploadDao dropboxUploadDao();

        // Метод для получения экземпляра базы данных
        public static AppDatabase getInstance(Context context) {
            if (INSTANCE == null) {
                synchronized (AppDatabase.class) {
                    if (INSTANCE == null) {

                        // Удаление старой базы данных перед созданием новой
                        // context.deleteDatabase("terminal_database");

                        INSTANCE = Room.databaseBuilder(context.getApplicationContext(),
                            AppDatabase.class, "device_database")
                            .fallbackToDestructiveMigration() // Очистка старой базы данных
                            .build();
                    }
                }
            }
            return INSTANCE;
        }
    }
}

```

Device.java:

// Аннотация @Entity указывает, что класс представляет таблицу в базе данных Room.

```

@Entity(tableName = "device")
public class Device {
    @PrimaryKey(autoGenerate = true)
    public int id;
    private String name;
    private String address;
    private long timestamp;

    public Device(String name, String address, long timestamp) {
        this.name = name;
        this.address = address;
        this.timestamp = timestamp;
    }

    // Геттеры и сеттеры для name, address, и timestamp
    public String getName() {

```

```

        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    public long getTimestamp() {
        return timestamp;
    }

    public void setTimestamp(long timestamp) {
        this.timestamp = timestamp;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getFormattedTimestamp() {
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
        Date date = new Date(timestamp);
        return sdf.format(date);
    }
}

```

DeviceDao:

```

@Dao
public interface DeviceDao {

    // Добавление устройства в базу данных
    @Insert
    void insert(Device device);

    // Получение всех устройств
    @Query("SELECT * FROM device")
    List<Device> getAllDevices();

    // Получение устройства по адресу
    @Query("SELECT * FROM device WHERE address = :address LIMIT 1")
    Device getDeviceByAddress(String address);

    // Удаление устройства
    @Delete
    void delete(Device device);

    // Очистка всей таблицы устройств
}

```

```

    @Query("DELETE FROM device")
    void deleteAll();
}

```

DatabasePagerAdapter.java:

```

public class DatabasePagerAdapter extends FragmentStateAdapter {

    public DatabasePagerAdapter(@NonNull Fragment fragment) {
        super(fragment);
    }

    @NonNull
    @Override
    public Fragment createFragment(int position) {
        switch (position) {
            case 0:
                return new DevicesFragment();
            case 1:
                return new MessagesFragment();
            case 2:
                return new LogsFragment();
            case 3:
                return new DropboxUploadsFragment();
            default:
                return new DevicesFragment();
        }
    }

    @Override
    public int getItemCount() {
        return 4; // Четыре таблицы
    }
}

```

DeviceFragment.java:

```

public class DevicesFragment extends Fragment {
    private AppDatabase database;
    private LinearLayout containerLayout;

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_devices, container, false);

        // Инициализация базы данных
        database = AppDatabase.getInstance(requireContext());

        // Найдём элементы UI
        Button loadButton = view.findViewById(R.id.loadButton);
        Button clearButton = view.findViewById(R.id.clearButton);
        containerLayout = view.findViewById(R.id.containerLayout);

        // Кнопка "Загрузить устройства"
        loadButton.setOnClickListener(v -> loadData());

        // Кнопка "Очистить базу"
        clearButton.setOnClickListener(v -> clearDatabase());
    }
}

```

```

        return view;
    }

    // Метод загрузки данных из базы
    private void loadData() {
        new Thread() -> {
            List<Device> devices = database.deviceDao().getAllDevices();
            getActivity().runOnUiThread() -> {
                containerLayout.removeAllViews();
                for (Device device : devices) {
                    // Форматируем строку, чтобы включить id устройства
                    String deviceInfo = "ID: " + device.getId() + " | " + device.getName() + " - " + device.getAddress()
                        + " | Добавлено: " + device.getFormattedTimestamp();

                    // Создаем TextView для отображения информации о каждом устройстве
                    TextView textView = new TextView(getContext());
                    textView.setText(deviceInfo);
                    textView.setTextSize(16);
                    textView.setPadding(8, 8, 8, 8);
                    containerLayout.addView(textView);
                }
            });
        }.start();
    }

    // Очистка базы данных
    private void clearDatabase() {
        new Thread() -> {
            try {
                // Удаляем все записи из таблицы логов перед очисткой таблицы устройств
                database.logEntryDao().deleteAll(); // Очистить таблицу логов

                // Теперь можно безопасно удалить все записи из таблицы устройств
                database.deviceDao().deleteAll(); // Очистить таблицу устройств

                // Обновляем UI
                getActivity().runOnUiThread() -> containerLayout.removeAllViews();
            } catch (Exception e) {
                Log.e("DevicesFragment", "Ошибка при очистке базы данных: ", e);
            }
        }.start();
    }
}

```

MessagesFragment.java:

```

public class MessagesFragment extends Fragment {

    private AppDatabase database;
    private LinearLayout containerLayout; // Контейнер для динамического добавления сообщений
    private Handler handler;

    // Конструктор фрагмента, привязывающий layout
    public MessagesFragment() {
        super(R.layout.fragment_messages);
    }

    // Токен доступа к Dropbox (должен храниться безопасно в продакшене!)
    private static final String ACCESS_TOKEN = "sl.u.AFpTtJSM..."; // Обрезано
}

```



```

@Override
public void onViewCreated(View view, Bundle savedInstanceState) {
    super.onViewCreated(view, savedInstanceState);

    // Привязка элементов интерфейса
    containerLayout = view.findViewById(R.id.containerLayout);
    Button loadButton = view.findViewById(R.id.loadButton);
    Button clearButton = view.findViewById(R.id.clearButton);

    handler = new Handler(); // Для обновления UI из фона
    database = AppDatabase.getInstance(requireContext());

    // Загрузка сообщений из базы данных
    loadButton.setOnClickListener(v -> loadData());

    // Очистка всех сообщений
    clearButton.setOnClickListener(v -> clearMessages());
}

/**
 * Загружает все сообщения из базы данных и добавляет их в интерфейс.
 */
private void loadData() {
    new Thread() -> {
        List<TerminalMessage> messages = database.terminalMessageDao().getAllMessages();
        handler.post() -> {
            containerLayout.removeAllViews();
            for (TerminalMessage message : messages) {
                addMessageToLayout(message);
            }
        });
    }.start();
}

/**
 * Добавляет сообщение с кнопками "Удалить" и "Dropbox" в интерфейс.
 */
private void addMessageToLayout(TerminalMessage message) {
    // Форматирование времени
    SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
    String formattedTime = sdf.format(new Date(message.timestamp));
    String fullMessage = formattedTime + " - " + message.message;

    // Контейнер для одного сообщения
    LinearLayout messageLayout = new LinearLayout(getContext());
    messageLayout.setOrientation(LinearLayout.HORIZONTAL);
    messageLayout.setPadding(0, 8, 0, 8);

    // Текст сообщения
    TextView messageTextView = new TextView(getContext());
    messageTextView.setText(fullMessage);
    messageTextView.setLayoutParams(new LinearLayout.LayoutParams(0,
        LinearLayout.LayoutParams.WRAP_CONTENT, 1));
    messageTextView.setMaxLines(Integer.MAX_VALUE);
    messageTextView.setEllipsize(null);
    messageTextView.setPadding(8, 8, 8, 8);
    messageTextView.setBreakStrategy(Layout.BREAK_STRATEGY_SIMPLE); // Обработка длинных слов

    // Кнопка "Удалить"

```

```

Button deleteButton = new Button(getContext());
deleteButton.setText("Удалить");
deleteButton.setTextSize(10f);
deleteButton.setLayoutParams(new LinearLayout.LayoutParams(200, 120));
deleteButton.setPadding(8, 8, 8, 8);
deleteButton.setOnClickListener(v -> deleteMessage(message));

// Кнопка "Dropbox"
Button uploadButton = new Button(getContext());
uploadButton.setText("Dropbox");
uploadButton.setTextSize(10f);
uploadButton.setLayoutParams(new LinearLayout.LayoutParams(200, 120));
uploadButton.setPadding(8, 8, 8, 8);
uploadButton.setOnClickListener(v -> uploadMessageToDropbox(message));

// Добавляем все элементы в строку
messageLayout.addView(messageTextView);
messageLayout.addView(deleteButton);
messageLayout.addView(uploadButton);

// Добавляем строку в контейнер
containerLayout.addView(messageLayout);
}

/**
 * Удаляет конкретное сообщение из базы данных.
 */
private void deleteMessage(TerminalMessage message) {
    new Thread() -> {
        database.terminalMessageDao().deleteById(message.id);
        handler.post(this::loadData); // Обновление UI
    }.start();
}

/**
 * Сохраняет сообщение во временный CSV-файл и передаёт его в Dropbox.
 */
private void uploadMessageToDropbox(TerminalMessage message) {
    new Thread() -> {
        File csvFile = new File(requireContext().getExternalFilesDir(null), "message_" + message.id + ".csv");

        // Создание временного CSV-файла
        try (FileWriter writer = new FileWriter(csvFile)) {
            writer.append("id,message,timestamp\n");
            writer.append(message.id + "," + message.message.replace(",", " ") + "," + message.timestamp + "\n");
            writer.flush();
        } catch (IOException e) {
            handler.post() -> Toast.makeText(getContext(), "Ошибка при создании файла",
Toast.LENGTH_SHORT).show());
            return;
        }

        // Начинаем загрузку
        uploadToDropbox(csvFile, message);
    }.start();
}

/**
 * Загружает файл в Dropbox и записывает результат в таблицу dropbox_uploads.

```

```

*/
private void uploadToDropbox(File file, TerminalMessage message) {
    new Thread() -> {
        try {
            // Настройка клиента Dropbox
            DbxRequestConfig config = DbxRequestConfig.newBuilder("dropbox/app").build();
            DbxClientV2 client = new DbxClientV2(config, ACCESS_TOKEN);

            // Открытие потока для передачи файла
            try (InputStream in = new FileInputStream(file)) {
                client.files().uploadBuilder("/") + file.getName())
                    .withMode(WriteMode.OVERWRITE)
                    .uploadAndFinish(in);

                // Сохраняем успешную загрузку в базе
                DropboxUpload uploadRecord = new DropboxUpload(message.id, System.currentTimeMillis(),
"success");
                database.dropboxUploadDao().insert(uploadRecord);

                handler.post() -> Toast.makeText(getApplicationContext(), "Файл загружен: " + file.getName(),
Toast.LENGTH_SHORT).show());
            }

        } catch (Exception e) {
            Log.e("DropboxError", "Ошибка загрузки: " + e.getMessage(), e);
            handler.post() -> Toast.makeText(getApplicationContext(), "Ошибка загрузки в Dropbox",
Toast.LENGTH_SHORT).show());
        }
    }).start();
}

/**
 * Очищает все сообщения из базы данных и обновляет UI.
 */
private void clearMessages() {
    new Thread() -> {
        database.terminalMessageDao().deleteAll();
        handler.post() -> {
            loadData();
            Toast.makeText(getApplicationContext(), "Сообщения очищены", Toast.LENGTH_SHORT).show();
        });
    }).start();
}
}

```

activity_main.xml:

```

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <!-- TabLayout для переключения вкладок -->
    <com.google.android.material.tabs.TabLayout
        android:id="@+id/tabLayout"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <!-- ViewPager2 для отображения содержимого вкладок -->

```

```

<androidx.viewpager2.widget.ViewPager2
    android:id="@+id/viewPager"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1" />

```

```

</LinearLayout>

```

fragment_bluetooth.xml:

```

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

```

```

    <Button
        android:id="@+id/bluetoothButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Включить Bluetooth" />

```

```

</LinearLayout>

```

fragment_device_scan.xml:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

```

```

    <!-- Список уже подключенных устройств -->

```

```

    <ListView
        android:id="@+id/connectedDevicesListView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:padding="8dp"
        android:dividerHeight="8dp" />

```

```

    <!-- Разделитель -->

```

```

    <View
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:background="#000000" />

```

```

    <!-- Секция сканирования устройств -->

```

```

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:orientation="vertical">

```

```

        <ProgressBar
            android:id="@+id/progressBar"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center"
            android:visibility="gone" />

```

```

    <ListView

```

```

        android:id="@+id/deviceListView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:padding="8dp"
        android:dividerHeight="8dp" />

<Button
    android:id="@+id/repeatScanButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Повторить поиск"
    android:layout_gravity="center" />

<Button
    android:id="@+id/sendMessageButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Разорвать соединение"
    android:layout_gravity="center" />
</LinearLayout>
</LinearLayout>

```

fragment_terminal.xml:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <!-- Поле ввода текста -->
    <EditText
        android:id="@+id/editTextInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Введите сообщение"
        android:inputType="text" />

    <!-- Кнопка для отправки сообщения -->
    <Button
        android:id="@+id/buttonSend"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Отправить" />

    <!-- ScrollView для отображения сообщений -->
    <ScrollView
        android:id="@+id/text_scroll_view"
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <LinearLayout
            android:id="@+id/messageContainer"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical" />
    </ScrollView>

```

```

<!-- Вывод текста -->
<TextView
    android:id="@+id/textViewOutput"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="16dp"
    android:text="Текст будет отображаться здесь" />

```

```

</LinearLayout>

```

fragment_database_view.xml:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <!-- TabLayout для переключения между таблицами -->
    <com.google.android.material.tabs.TabLayout
        android:id="@+id/tabLayout"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="8dp" />

    <!-- ViewPager2 для отображения фрагментов с таблицами -->
    <androidx.viewpager2.widget.ViewPager2
        android:id="@+id/viewPager"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />

```

```

</LinearLayout>

```

fragment_devices.xml:

```

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <!-- Кнопка для загрузки данных -->
    <Button
        android:id="@+id/loadButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Загрузить устройства" />

    <!-- Контейнер для вывода данных -->
    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:scrollbars="vertical">

        <LinearLayout
            android:id="@+id/containerLayout"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"

```

```
        android:orientation="vertical">
    </LinearLayout>

</ScrollView>

<!-- Кнопка для очистки данных -->
<Button
    android:id="@+id/clearButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Очистить устройства"
    android:layout_marginTop="8dp" />
</LinearLayout>
```

Приложение В

Акт о внедрении программного продукта, созданного по теме выпускной квалификационной работы Разработка мобильного приложения ComTerminal для обмена данными с устройствами по Bluetooth

Настоящий Акт свидетельствует, что мобильное приложение ComTerminal, разработанное Чистяковым Артёмом Александровичем, прошло внедрение на базе экспериментальной лаборатории взаимодействия с цифровыми измерительными приборами и мобильными устройствами.

Процесс внедрения программного средства занял 45 дней и проходил с 10 апреля по 25 мая 2025 года.

Характеристики и возможности программного средства:

- установка устойчивого Bluetooth-соединения с внешними устройствами;
- приём и отображение сообщений, полученных от прибора;
- сохранение сообщений во внутреннюю базу данных с возможностью дальнейшего анализа;
- ведение логов подключений, отключений;
- выборочная отправка сохранённых сообщений в облачное хранилище Dropbox;
- визуализация данных через структурированный и удобный пользовательский интерфейс.

Отличительные особенности приложения:

- высокая стабильность работы при взаимодействии с физическим оборудованием;
- простота и скорость использования;
- интеграция с облачными сервисами;
- локальное хранение истории сообщений и действий;
- кросс-устройственная совместимость на Android-платформах.

В ходе эксплуатации программного средства подтверждено, что все заявленные функциональные возможности были реализованы и соответствуют требованиям, поставленным на этапе проектирования.

Программа успешно справляется с приёмом, отображением и обработкой данных от Bluetooth-устройств и может использоваться в условиях реального применения — например, в образовательной, инженерной или производственной среде.

Руководитель
к.ф.-м.н., старший научный сотрудник ИГиЛ СО РАН
Ларичкин Алексей Юрьевич
01.06.2025г